

Neural Networks

Kaiqi Zhao

Harbin Institute of Technology, Shenzhen



Outline

- Learning from Observations: Artificial Neural Networks
- Backpropagation
- Non-linearity and Activation Functions
- Regularization
- Optimization

Symbolism v.s. Connectionism

Symbolism

Symbol manipulation

- **Advantages:**
 - Rigor
 - Interpretability
- **Limitations:**
 - Perception (e.g., recognizing images)
 - Need explicit programming of knowledge

Symbolism

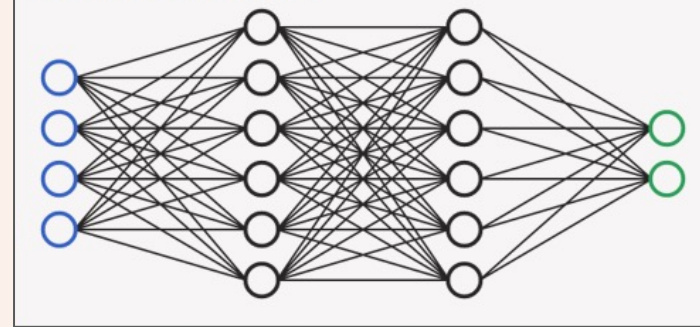
$$\begin{aligned}(p \rightarrow q) &\Leftrightarrow (\neg p \vee q) \\ &\Leftrightarrow (q \vee \neg p) \\ &\Leftrightarrow (\neg \neg q \vee \neg p) \\ &\Leftrightarrow (\neg q \rightarrow \neg p)\end{aligned}$$

Connectionism

Learning from observation

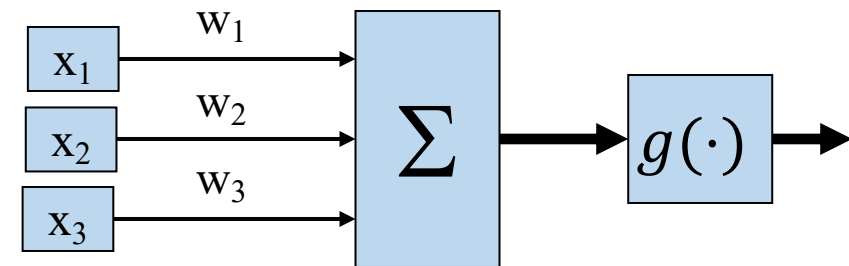
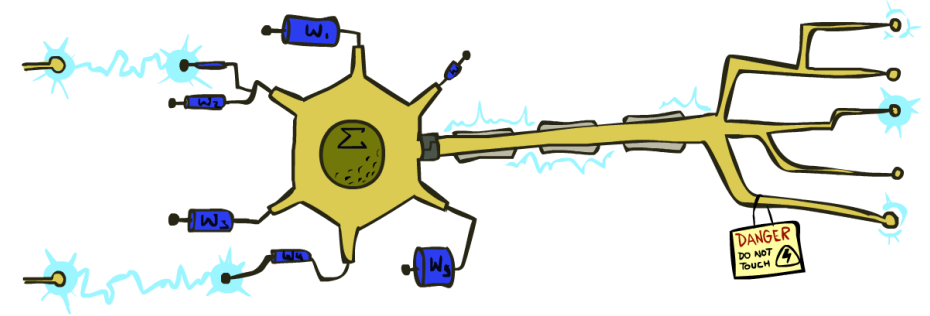
- **Advantages:**
 - Excel at perception
 - Learning instead of hard coding rules
- **Limitations:**
 - Less explainable, less rigor

Connectionism



Connectionism: Motivation from Neuroscience

- Biological Neural Network
 - A group of chemically connected or functionally associated **neurons**. A single neuron may be connected to many other neurons and the total number of neurons and connections in a network may be extensive
- An **Artificial Neuron** is composed of:
 - Input feature values
 - Weights for each input feature
 - Activation function $g(\cdot)$



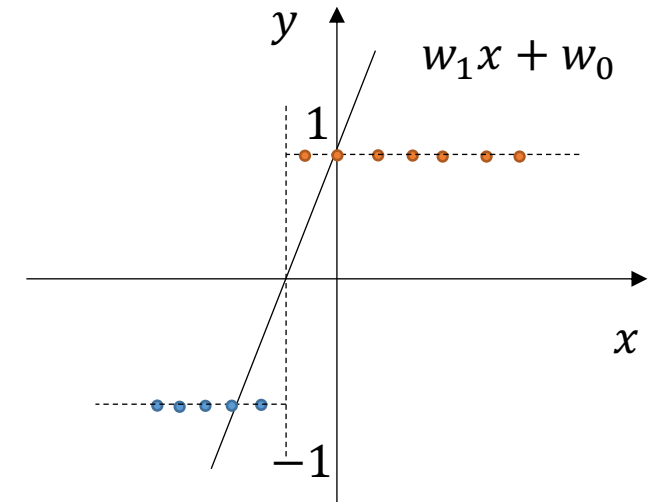
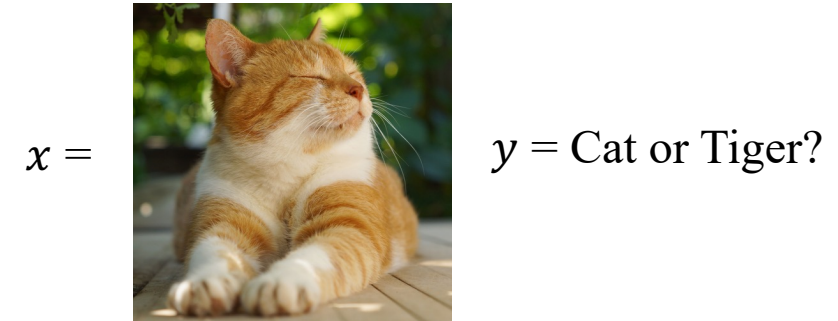
$$\hat{y} = g(w^T x) = g\left(\sum_i w_i x_i\right)$$

A Simple Linear Classification Model

- Represent the world as features
- Inputs are **feature values**, each feature has a **weight**
- A **sign activation** function applies to the weighted sum of the features, which outputs discrete class values

$$\hat{y} = \text{Sign}(w^T x) = \text{Sign}\left(\sum_i w_i x_i\right)$$

- This is an artificial neuron!



Learning from observations

- We need to define an evaluation function / loss function to evaluate how good the weights are
- For each observation (sample), we can compare the predicted value $\hat{y}$ and the actual class label y

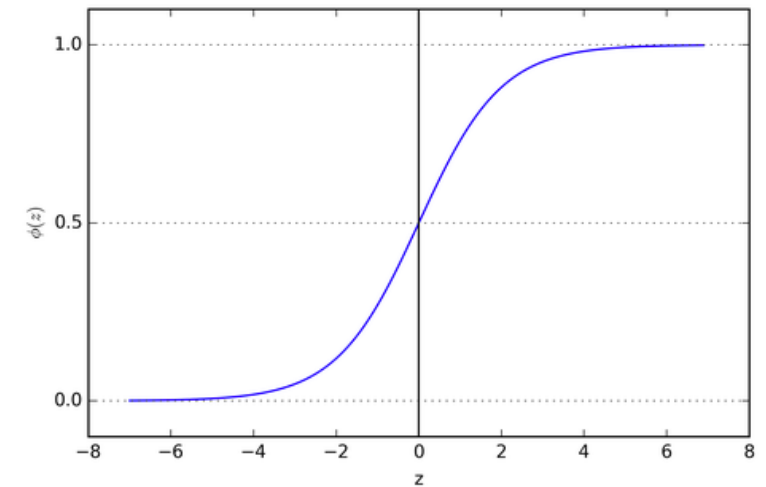
$$\mathcal{L}(w) = \sum_{(x^{(i)}, y^{(i)}) \in D} d(\hat{y}^{(i)}, y^{(i)}) = \sum_{(x^{(i)}, y^{(i)}) \in D} d(\text{Sign}(w^T x^{(i)}), y^{(i)})$$

- We have learned from the lectures of local search that:
 - This is a minimization problem with continuous variables
 - We can use gradient descent!

Sigmoid Function

- The sign function $Sign(x)$ is not differentiable at $x = 0$
- Consider the problem as a probabilistic classification problem – we output a probability value of each class label provided $w^T x$ instead of the exact class labels
- We want
 - If $z = w^T x$ is very positive $\rightarrow$ probability goes to 1
 - If $z = w^T x$ is very negative $\rightarrow$ probability goes to 0
- Use the Sigmoid (logistic) function on z

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$



Logistic Regression

- Given that the sigmoid function outputs value within $[0, 1]$, it could be considered as **the probability of positive label** for a binary classification problem.
- We can use maximum likelihood estimation – find the parameters w which maximizes the probability of all training examples being classified correctly:

$$w_{ML} = \operatorname{argmax}_w \log \mathcal{L}(w) = \operatorname{argmax}_w \sum_i \log P(y^{(i)} | x^{(i)}; w)$$

with

$$P(y^{(i)} = +1 | x^{(i)}; w) = \frac{1}{1 + e^{-w^T x^{(i)}}}$$

$$P(y^{(i)} = -1 | x^{(i)}; w) = 1 - \frac{1}{1 + e^{-w^T x^{(i)}}} = \frac{e^{-w^T x^{(i)}}}{1 + e^{-w^T x^{(i)}}}$$

= Logistic Regression

Example – Prediction

- Let's consider the following data set of 1 feature. Let the weights $w_0 = 1, w_1 = 2$ be the best you get from maximum likelihood

i	x	y
1	-2	-1
2	0	+1
...	...	...

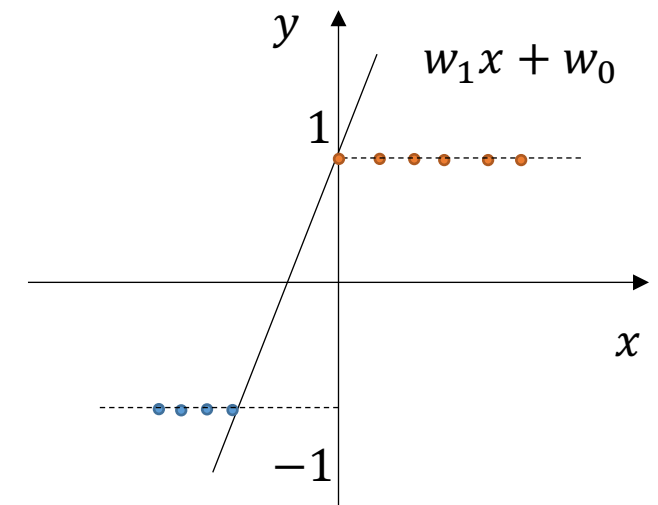
- Make prediction by calculating the probability:

$$P(y^{(1)} = +1 | x^{(1)}; w) = \frac{1}{1 + e^{-(-2*2+1)}} = \frac{1}{1 + e^3} \approx 4.7\%$$

$$P(y^{(1)} = -1 | x^{(1)}; w) = 1 - P(y^{(1)} = +1 | x^{(1)}; w) \approx 95.3\%$$

$$P(y^{(2)} = +1 | x^{(2)}; w) = \frac{1}{1 + e^{-1}} \approx 73.1\%$$

$$P(y^{(2)} = -1 | x^{(2)}; w) = 1 - P(y^{(2)} = +1 | x^{(2)}; w) \approx 26.9\%$$



$$P(y^{(i)} = +1 | x^{(i)}; w) = \frac{1}{1 + e^{-w^T x^{(i)}}}$$

Logistic Regression

- The previous definition of Logistic Regression treats the two classes differently. In fact, you could also define it with its equivalent form:

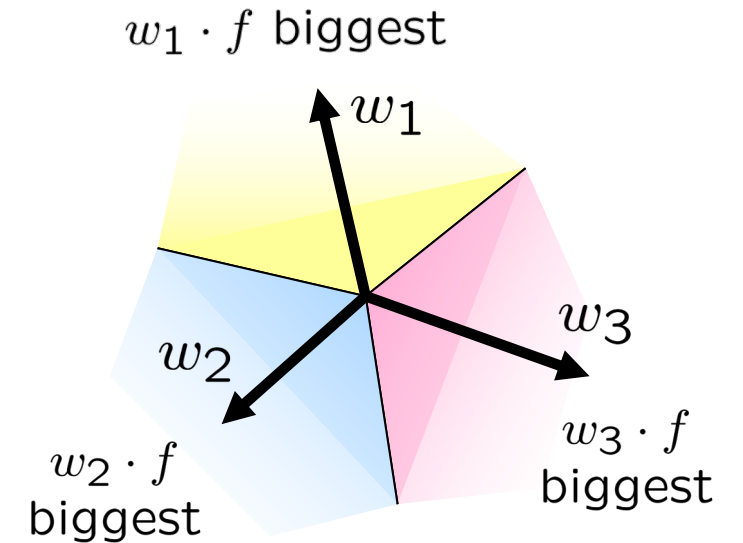
$$P(y^{(i)} = +1|x^{(i)}; w_1, w_2) = \frac{e^{w_1^T x^{(i)}}}{e^{w_1^T x^{(i)}} + e^{w_2^T x^{(i)}}}$$

$$P(y^{(i)} = -1|x^{(i)}; w_1, w_2) = \frac{e^{w_2^T x^{(i)}}}{e^{w_1^T x^{(i)}} + e^{w_2^T x^{(i)}}}$$

- In this way, the two classes are treated **in the same way**
 - We have one weight vector for each class (w_1 for class +1, and w_2 for class -1)
 - The conditional probability of each class only depend on the feature $x^{(i)}$ and its weight, the denominator is only a normalization term.
 - With this form, it is easier for us to extend Logistic Regression to multi-class problems.

Multiclass Logistic Regression

- Multi-class linear classification
 - A weight vector for each class: w_c
 - Score for a class y : $w_c^T x$
 - Prediction with the highest score: $y = \operatorname{argmax}_{c \in C} w_c^T x$
- Suppose we have class labels $\{1, 2, 3\}$
- How to make the scores into probabilities?
 - Extend the binary logistic regression
 - Let $z_1 = w_1^T x$, $z_2 = w_2^T x$, $z_3 = w_3^T x$



$$\underbrace{z_1, z_2, z_3}_{\text{original outputs}} \rightarrow \underbrace{\frac{e^{z_1}}{e^{z_1} + e^{z_2} + e^{z_3}}, \frac{e^{z_2}}{e^{z_1} + e^{z_2} + e^{z_3}}, \frac{e^{z_3}}{e^{z_1} + e^{z_2} + e^{z_3}}}_{\text{Softmax transformation}}$$

Multiclass Logistic Regression

- Maximum likelihood estimation:

$$w_{ML} = \operatorname{argmax}_w \log \mathcal{L}(w) = \operatorname{argmax}_w \sum_i \log P(y^{(i)} | x^{(i)}; w)$$

with

$$P(y^{(i)} = c | x^{(i)}; w) = \frac{e^{w_c x^{(i)}}}{\sum_{c'} e^{w_{c'} x^{(i)}}}$$

= Multi-Class Logistic Regression

Finding the Weights of Logistic Regression

- Optimization
 - i.e., how do we solve:

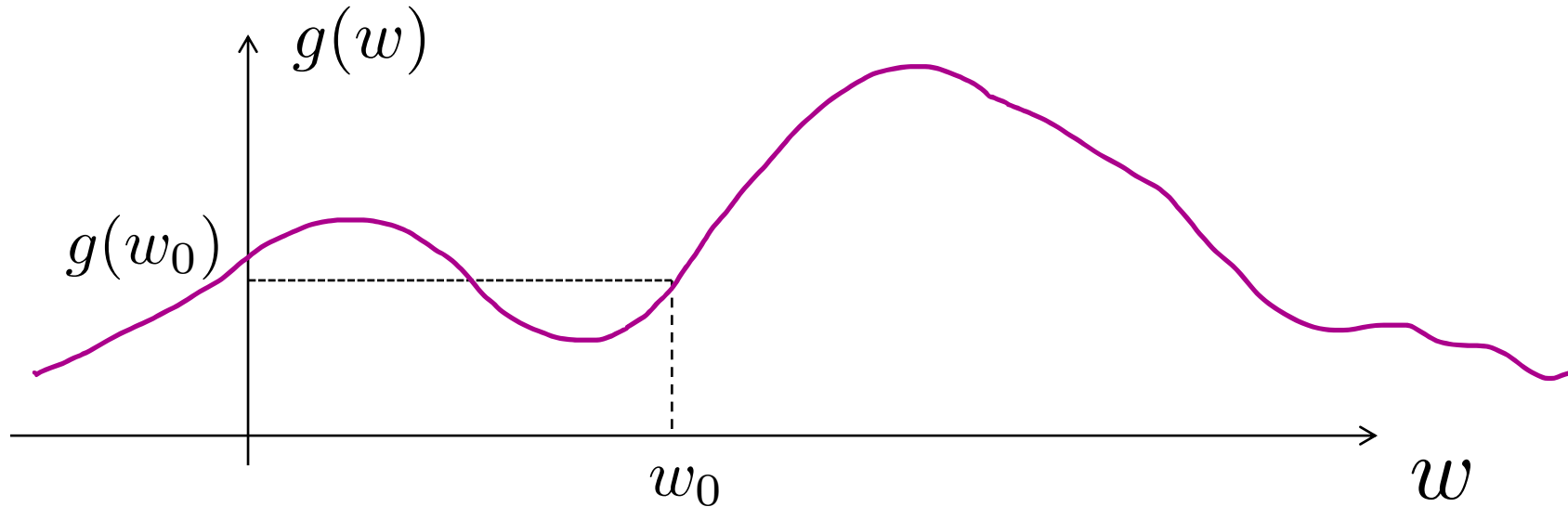
$$w_{ML} = \operatorname{argmax}_w \log \mathcal{L}(w) = \operatorname{argmax}_w \sum_i \log P(y^{(i)} | x^{(i)}; w)$$

Hill Climbing

- Local search:
 - Start wherever
 - Repeat: move to the best neighboring state
 - If no neighbors better than current, quit
- What's particularly tricky when hill-climbing for multi-class logistic regression?
 - Optimization over a continuous space
 - Infinitely many neighbors!
 - How to do this efficiently?
 - **Select the direction which is steepest to move!**



Finding the Steepest Direction | 1-D Optimization



- Could evaluate $g(w_0 + h)$ and $g(w_0 - h)$
 - Then step in best direction

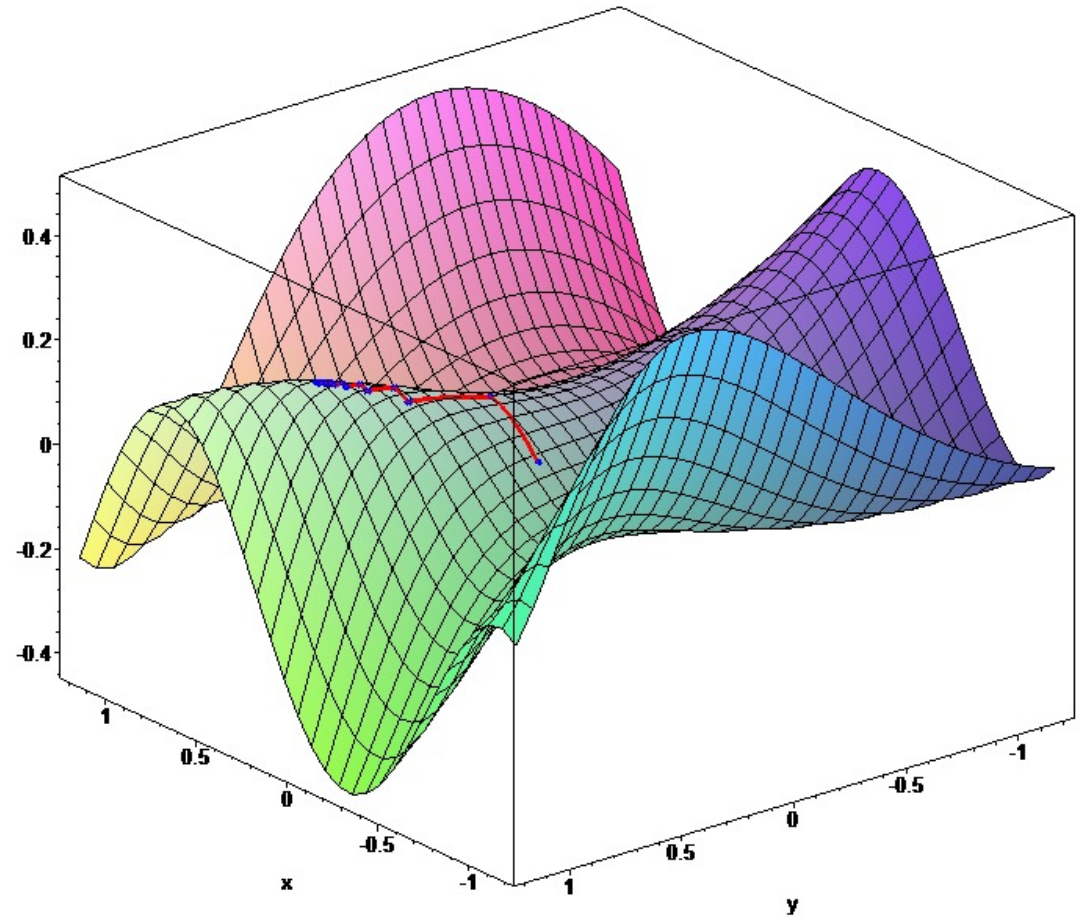
- Or, evaluate derivative (slope):
$$\frac{\partial g(w_0)}{\partial w} = \lim_{h \rightarrow 0} \frac{g(w_0 + h) - g(w_0 - h)}{2h}$$

Finding the Steepest Direction | 2-D Optimization

Gradient provides the steepest direction in a 2D or higher dimensional optimization problem.

Let the two parameters to be estimated be w_1 and w_2 , and $g(w_1, w_2)$ be the function you want to maximize/minimize. Then, the gradient at $w_1 = a, w_2 = b$ is

$$\nabla g(a, b) = \begin{bmatrix} \frac{\partial g}{\partial w_1}(a, b) \\ \frac{\partial g}{\partial w_2}(a, b) \end{bmatrix}$$



Gradient Ascent

- Perform update in uphill direction for each coordinate
- The steeper the slope (i.e. the higher the derivative) the bigger the step for that coordinate
- E.g., consider: $g(w_1, w_2)$, let $w_1 = a$ and $w_2 = b$ currently

Updates:

$$w_1 \leftarrow a + \alpha * \frac{\partial g}{\partial w_1}(a, b)$$

$$w_2 \leftarrow b + \alpha * \frac{\partial g}{\partial w_2}(a, b)$$

α is learning rate that controls how fast we step

Updates: in vector notation:

$$w \leftarrow w + \alpha * \nabla_w g(a, b)$$

$$\text{with } \nabla_w g(a, b) = \begin{bmatrix} \frac{\partial g}{\partial w_1}(a, b) \\ \frac{\partial g}{\partial w_2}(a, b) \end{bmatrix} = \text{gradient}$$

Gradient Ascent

- Idea:
 - Start somewhere
 - Repeat: Take a step in the gradient direction

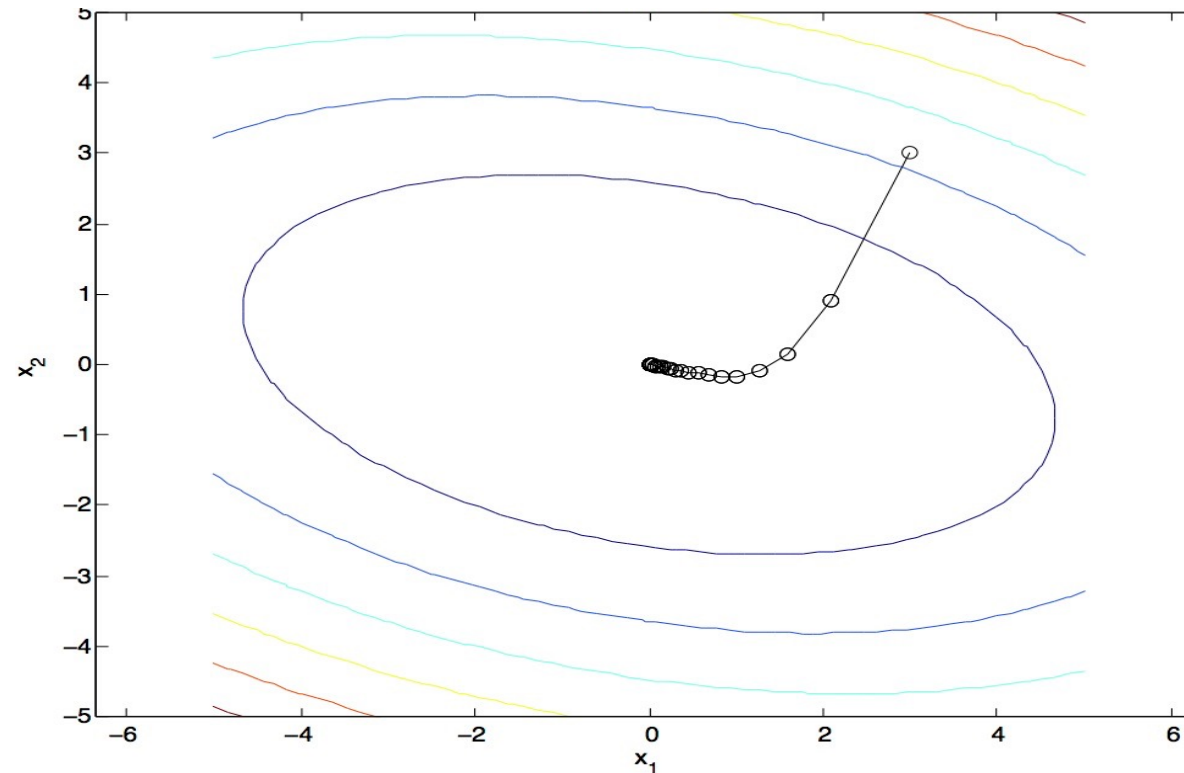


Figure source: Mathworks

Optimization Procedure: Gradient Ascent

- init $w = w_0$
- for iteration $t = 1, 2, \dots$

$$w_t \leftarrow w_{t-1} + \alpha * \nabla g(w_{t-1})$$

- α : learning rate – the parameter that controls how fast we climb the hill
- A small learning rate – slow learning
- A large learning rate – might miss the peak!

Batch Gradient Ascent on the Log Likelihood Objective

$$\max_w \log \mathcal{L}(w) = \max_w \underbrace{\sum_i \log P(y^{(i)} | x^{(i)}; w)}_{g(w)}$$

- init $w = w_0$
- for iteration $t = 1, 2, \dots$

$$w_t \leftarrow w_{t-1} + \alpha * \sum_i \nabla \log P(y^{(i)} | x^{(i)}; w_{t-1})$$

Stochastic Gradient Ascent on the Log Likelihood Objective

$$\max_w \log \mathcal{L}(w) = \max_w \sum_i \log P(y^{(i)} | x^{(i)}; w)$$

SGD: once gradient on **one training example** has been computed, incorporate before computing next one. This is an approximation to the full gradient.

- init $w = w_0$
- for iteration $t = 1, 2, \dots$
 - **pick random training example j**
$$w_t \leftarrow w_{t-1} + \alpha * \nabla \log P(y^{(j)} | x^{(j)}; w_{t-1})$$

Stochastic Gradient Ascent on the Log Likelihood Objective

$$\max_w \log \mathcal{L}(w) = \max_w \sum_i \log P(y^{(i)} | x^{(i)}; w)$$

Mini-batch: gradient over **small sets of training examples** can be computed in parallel, instead of a single one. A better approximation to full gradient

- init $w = w_0$
- for iteration $t = 1, 2, \dots$
 - pick random **subset** of training examples J

$$w_t \leftarrow w_{t-1} + \alpha * \sum_{j \in J} \nabla \log P(y^{(j)} | x^{(j)}; w_{t-1})$$

Stochastic Gradient Ascent on the Log Likelihood Objective

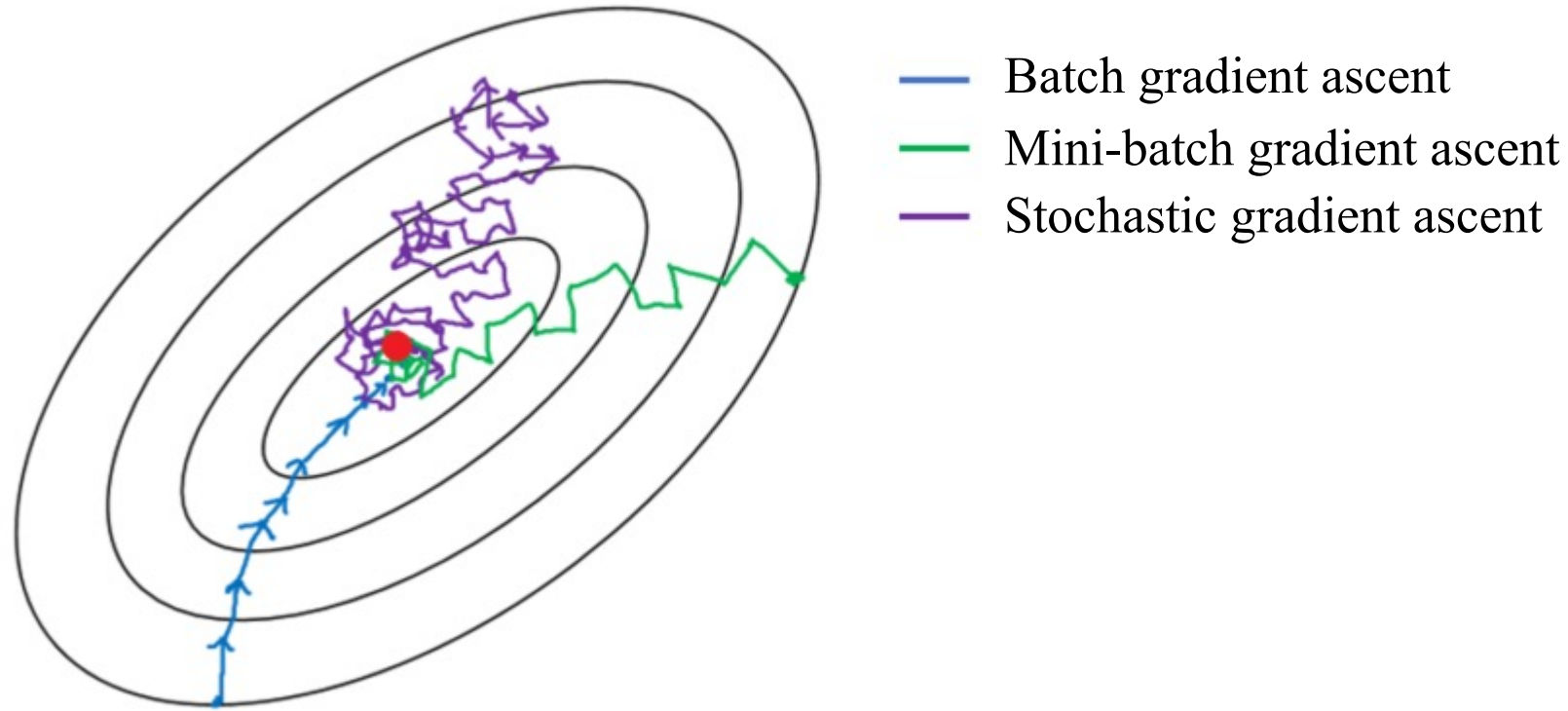


Image adapted from: <https://sweta-nit.medium.com/batch-mini-batch-and-stochastic-gradient-descent-e9bc4cacd461>

Cross-Entropy Loss

- In maximum likelihood estimation, we optimize

$$w_{ML} = \operatorname{argmax}_w \log \mathcal{L}(w) = \operatorname{argmax}_w \sum_i \log P(y^{(i)} | x^{(i)}; w)$$

- Let $[y_1^{(i)}, y_2^{(i)}, \dots, y_C^{(i)}]^T$ be a one-hot vector, where $y_k^{(i)} = 1$ if the class label of the i -th instance is k , i.e., $y^{(i)} = k$, otherwise, $y_k^{(i)} = 0$
- We can rewrite the likelihood: $\mathcal{L}(w) = \prod_i \prod_k P(k | x^{(i)}; w)^{y_k^{(i)}}$
- Thus, maximizing the log-likelihood is equivalent to minimize the loss (**Cross-Entropy**)

$$CE(w) = -\log \mathcal{L}(w) = -\sum_i \sum_k y_k^{(i)} \log P(k | x^{(i)}; w)$$

Example

- We have two different weight vectors (w_1 and w_2) results in the following predictions of weather (sunny, rain, windy):

i	Predictions with w_1 (S, R, W)	Predictions with w_2 (S, R, W)	y (S, R, W)
1	0.7, 0.2, 0.1	0.4, 0.3, 0.3	1, 0, 0
2	0.1, 0.1, 0.8	0.2, 0.1, 0.7	0, 0, 1
3	0.3, 0.3, 0.4	0.2, 0.2, 0.6	0, 1, 0

$$CE(w) = - \sum_i \sum_k y_k^{(i)} \log P(k|x^{(i)}; w)$$

Weight vector w_1

Sample 1 loss:

$$-1 * \log 0.7 - 0 * \log 0.2 - 0 * \log 0.1 \approx 0.357$$

Sample 2 loss:

$$-0 * \log 0.1 - 0 * \log 0.1 - 1 * \log 0.8 \approx 0.223$$

Sample 3 loss:

$$-0 * \log 0.3 - 1 * \log 0.3 - 0 * \log 0.4 \approx 1.201$$

$$CE(w_1) = 0.357 + 0.223 + 1.201 = 1.781$$

Weight vector w_2

Sample 1 loss: $-1 * \log 0.4 \approx 0.916$

Sample 2 loss: $-1 * \log 0.7 \approx 0.357$

Sample 3 loss: $-1 * \log 0.2 \approx 1.609$

$$CE(w_2) = 0.916 + 0.357 + 1.609 = 2.882$$

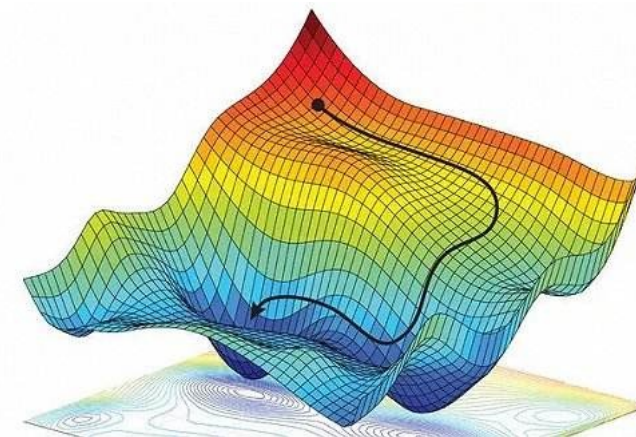
Gradient Descent

$$\operatorname{argmin}_w CE(w) = \operatorname{argmin}_w - \sum_i \sum_k y_k^{(i)} \log P((k|x^{(i)}; w)$$

- To minimize cross-entropy, we can use gradient descent – going downhill
- Gradient descent goes towards the opposite direction of gradient ascent

$$w \leftarrow w - \alpha * \nabla CE(w)$$

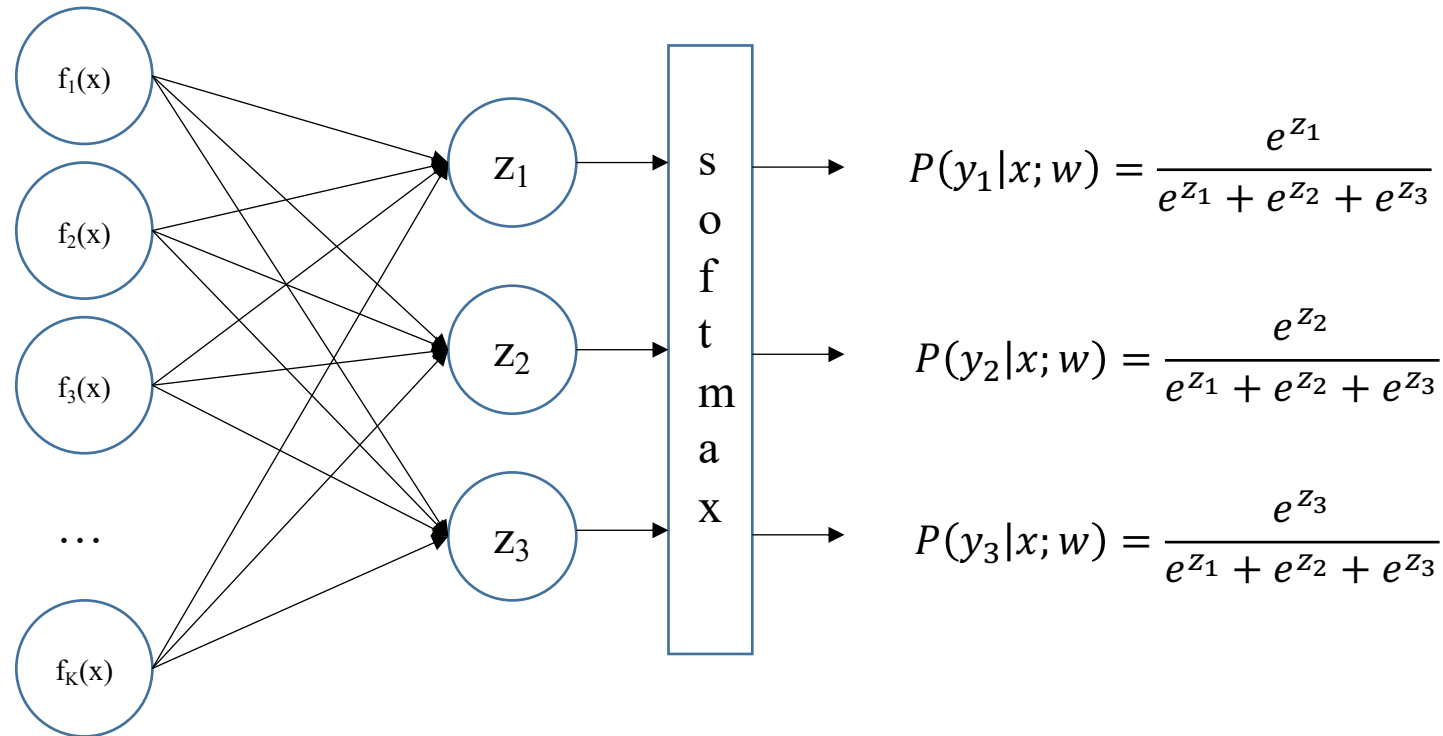
- Variants:
 - Stochastic Gradient Descent (SGD)
 - Mini-batch SGD



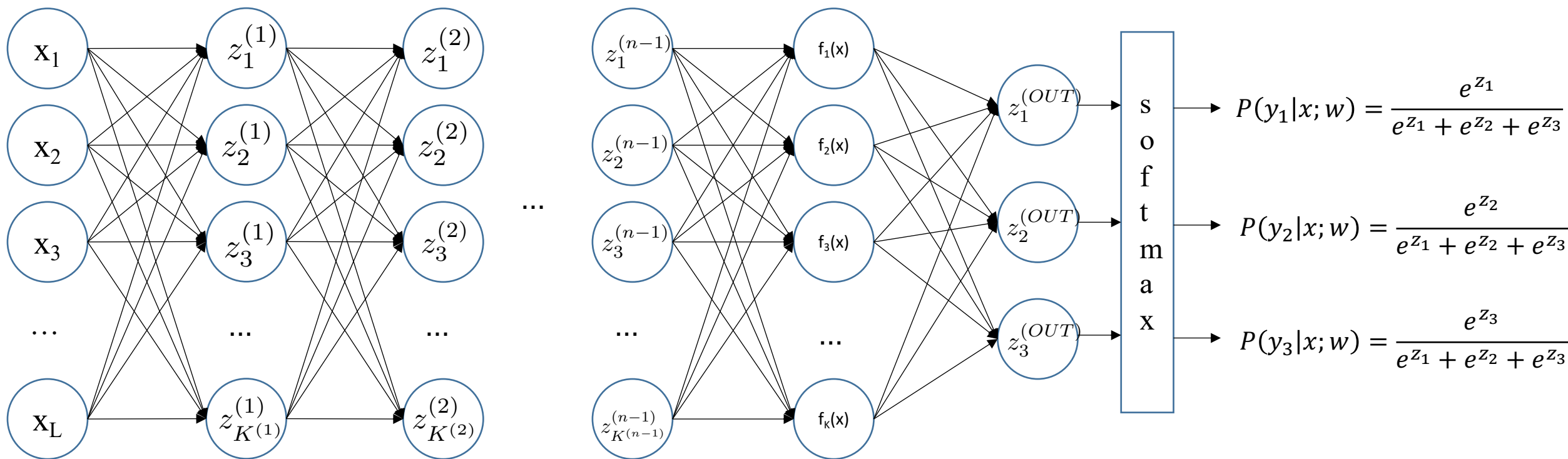
Neural Networks

Multi-class Logistic Regression

- Multi-class Logistic Regression is **a special case of neural network**



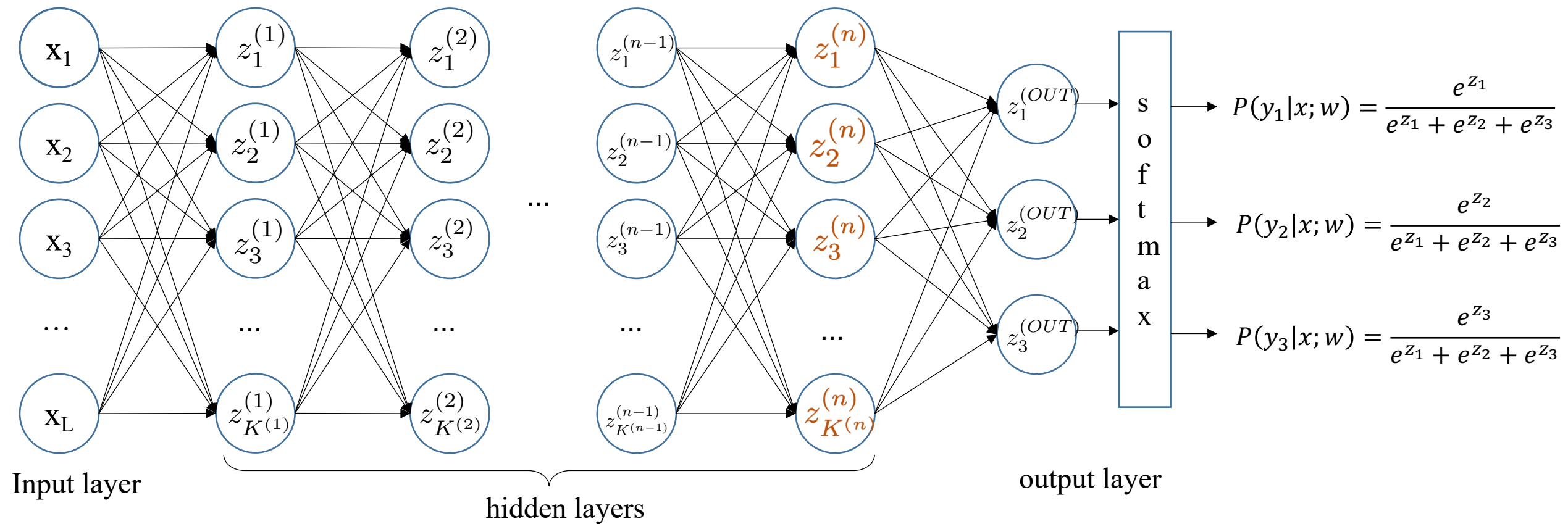
Neural Network



$$z_i^{(k)} = g\left(\sum_j W_{i,j}^{(k-1,k)} z_j^{(k-1)}\right)$$

g = nonlinear activation function

Neural Network



$$z_i^{(k)} = g\left(\sum_j W_{i,j}^{(k-1,k)} z_j^{(k-1)}\right)$$

g = nonlinear activation function

Intuitive Example

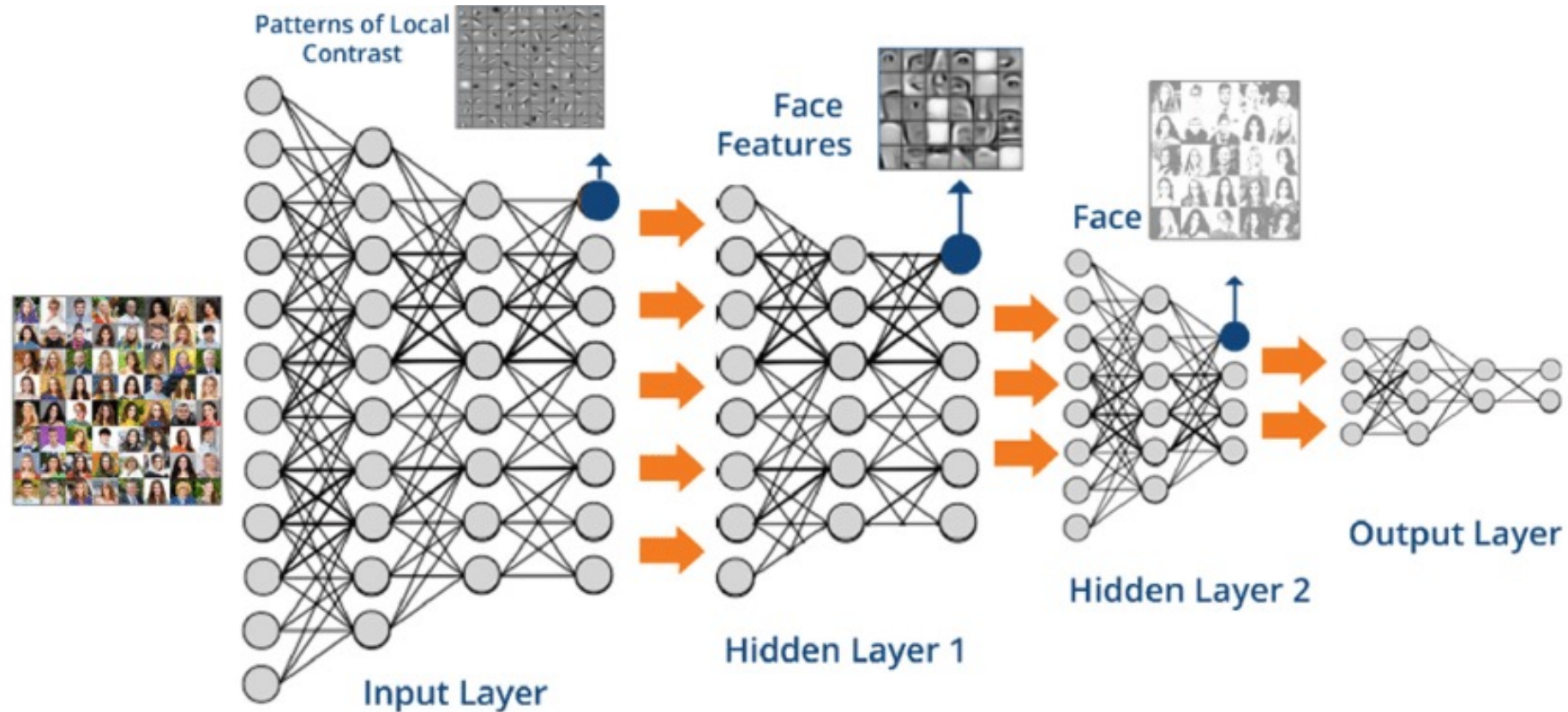
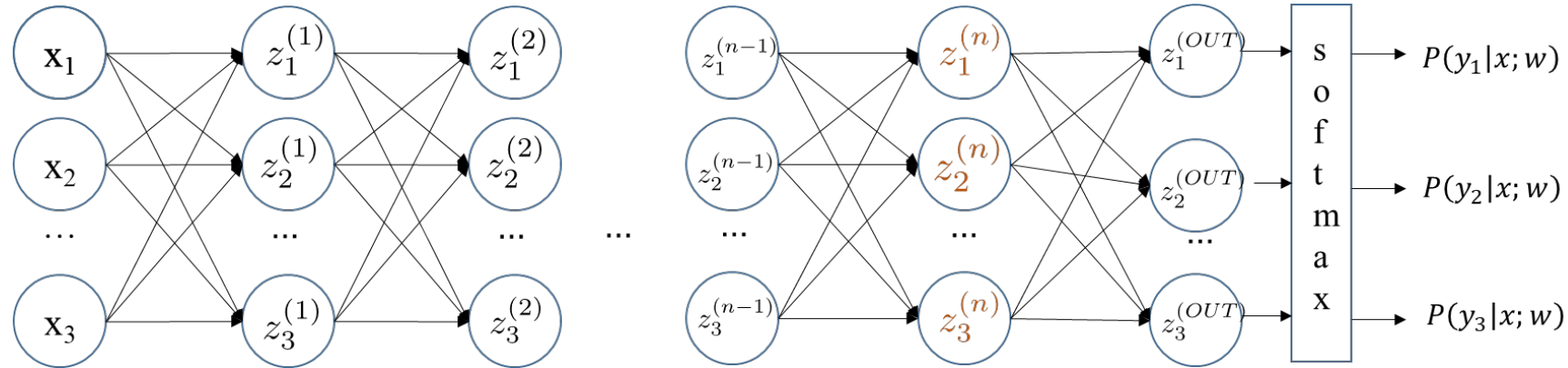


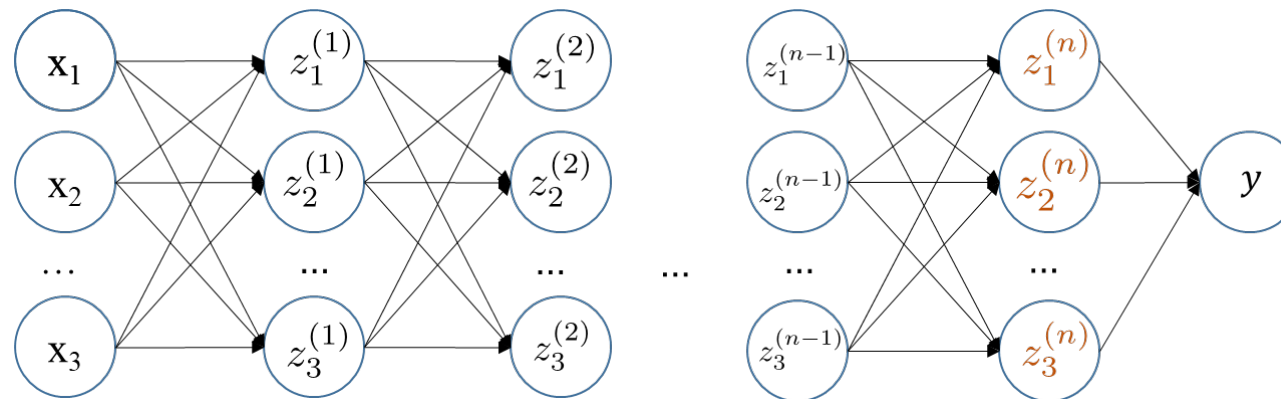
Image source: Grigsby, Scott. (2018). Artificial Intelligence for Advanced Human-Machine Symbiosis.
10.1007/978-3-319-91470-1_22.

Neural Network

Neural Networks for classification



Neural Networks for regression

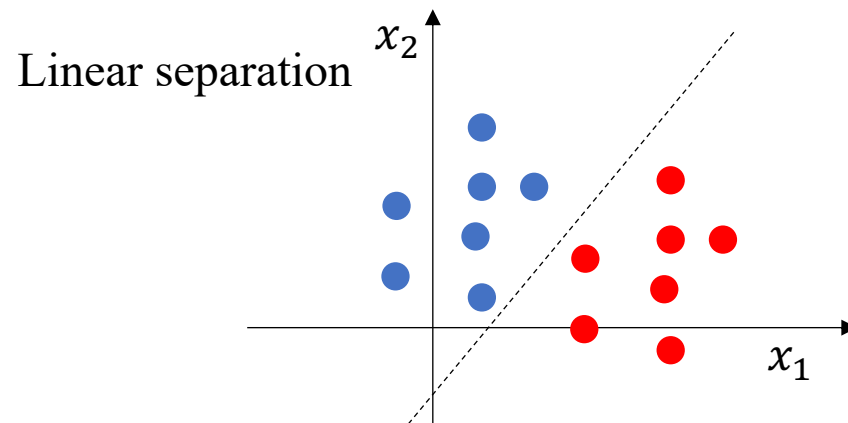
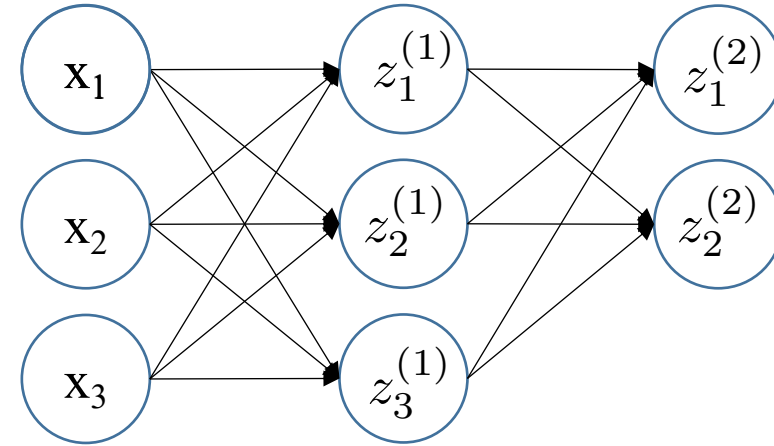


$$z_i^{(k)} = g\left(\sum_j W_{i,j}^{(k-1,k)} z_j^{(k-1)}\right)$$

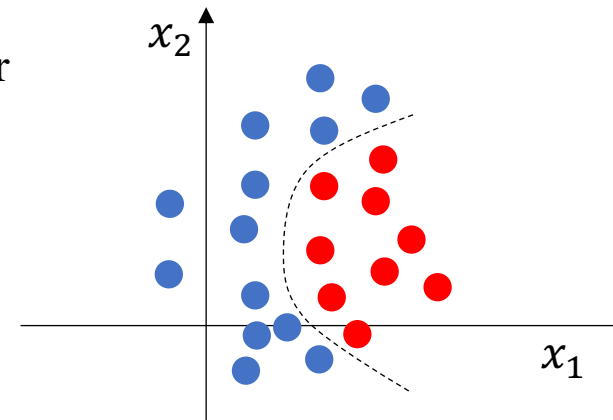
g = nonlinear activation function

Why Non-linear Activation?

- How if we stack several linear layers?
 - For example two layers:
 - $z^{(1)} = w_1^T x$
 - $z^{(2)} = w_2^T z^{(1)}$
 - $z^{(2)} = w_2^T w_1^T x$
- It is equivalent to a single linear layer!!
- Some datasets are not linearly separable!!



Non-Linear separation



Training a Neural Network

- Training a neural network for classification is just like logistic regression:
 - Maximize the log-likelihood, or
 - Minimize the cross-entropy loss
- Training a neural network for regression is to minimize the sum of square error
- Run gradient ascent/descent! → we need to compute the derivatives

Computing the derivatives

- Computing the derivatives for complex objective functions - **CHAIN RULE**:

If $f(x) = g(h(x))$

Then
$$\frac{df(x)}{dx} = \frac{dg}{dh} \cdot \frac{dh(x)}{dx}$$

→ Derivatives can be computed by following well-defined procedures

Example

- The cross-entropy for binary classification problem:

$$CE(w) = -y \ln \hat{y} - (1 - y) \ln(1 - \hat{y}), \quad \hat{y} = p(y = +1|x, w) = \frac{1}{1 + e^{-wx}}$$

- By chain rule:

$$\frac{dCE(w)}{dw} = \frac{dCE(w)}{d\hat{y}} \cdot \frac{d\hat{y}}{dw}$$

$$\frac{dCE(w)}{d\hat{y}} = -\frac{y}{\hat{y}} + \frac{1-y}{1-\hat{y}} = \frac{\hat{y}-y}{\hat{y}(1-\hat{y})} \quad \frac{d\hat{y}}{dw} = ?$$

$$\text{Let } h = -wx, \text{ then } \hat{y} = \frac{1}{1+e^h}$$

$$\frac{d\hat{y}}{dw} = \frac{d\hat{y}}{dh} \cdot \frac{dh}{dw} = -\frac{e^h}{(1+e^h)^2} \cdot -x = \hat{y}(1-\hat{y})x$$

$$\frac{dCE(w)}{dw} = \frac{dCE(w)}{d\hat{y}} \cdot \frac{d\hat{y}}{dw}$$

$$= \frac{\hat{y}-y}{\hat{y}(1-\hat{y})} \cdot \hat{y}(1-\hat{y})x$$

$$= (\hat{y}-y)x$$

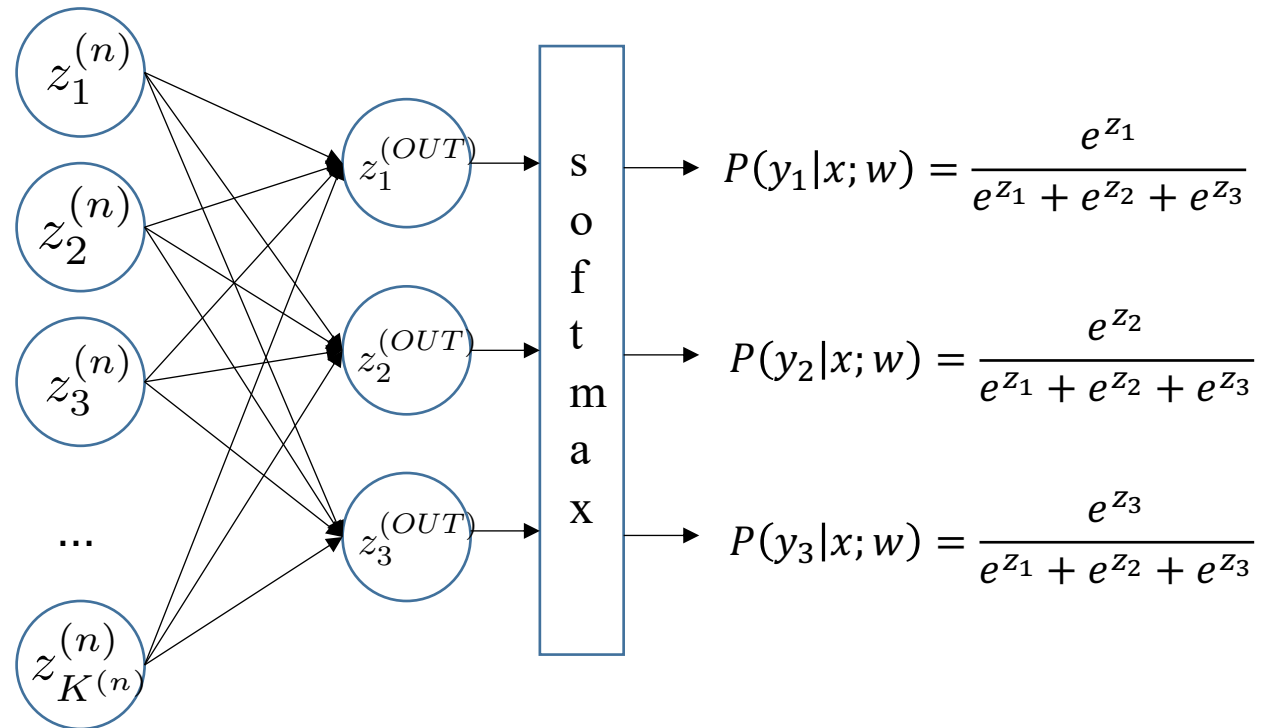
Gradient descent:

$$w \leftarrow w - \alpha \cdot (\hat{y} - y)x$$

Training a Neural Network

How if we have a complicated network:

- Forward
- Backprop



g = nonlinear activation function

Training a Neural Network

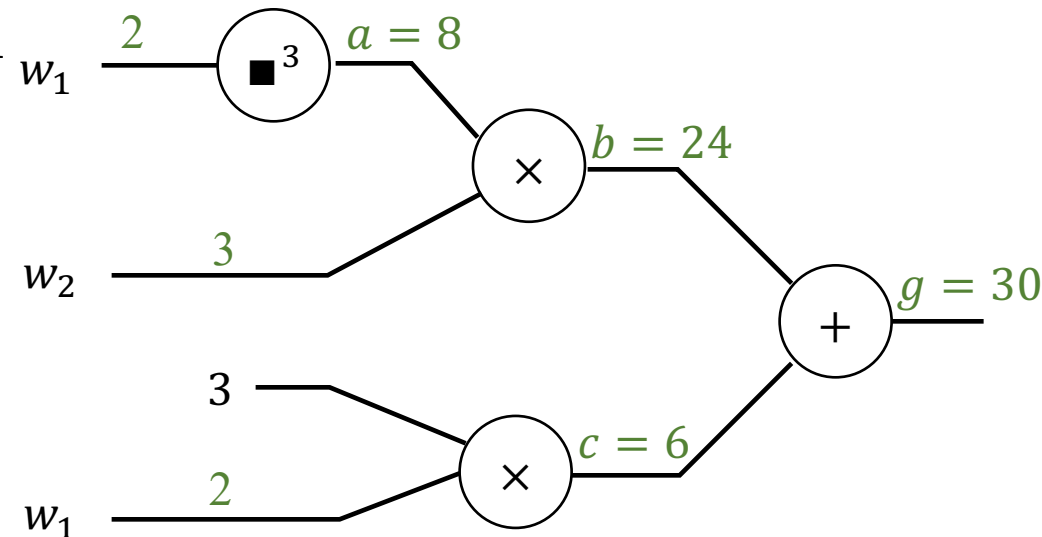
- Let's use a simple example to learn how to train a Neural Network.
- Basically, the loss function (e.g., cross-entropy) of a Neural Network can be considered as a complex function that maps your parameters w to the loss.

- **A simple (loss) function:** $g(w) = w_1^3 w_2 + 3w_1$

- A computational graph could be constructed from the objective function

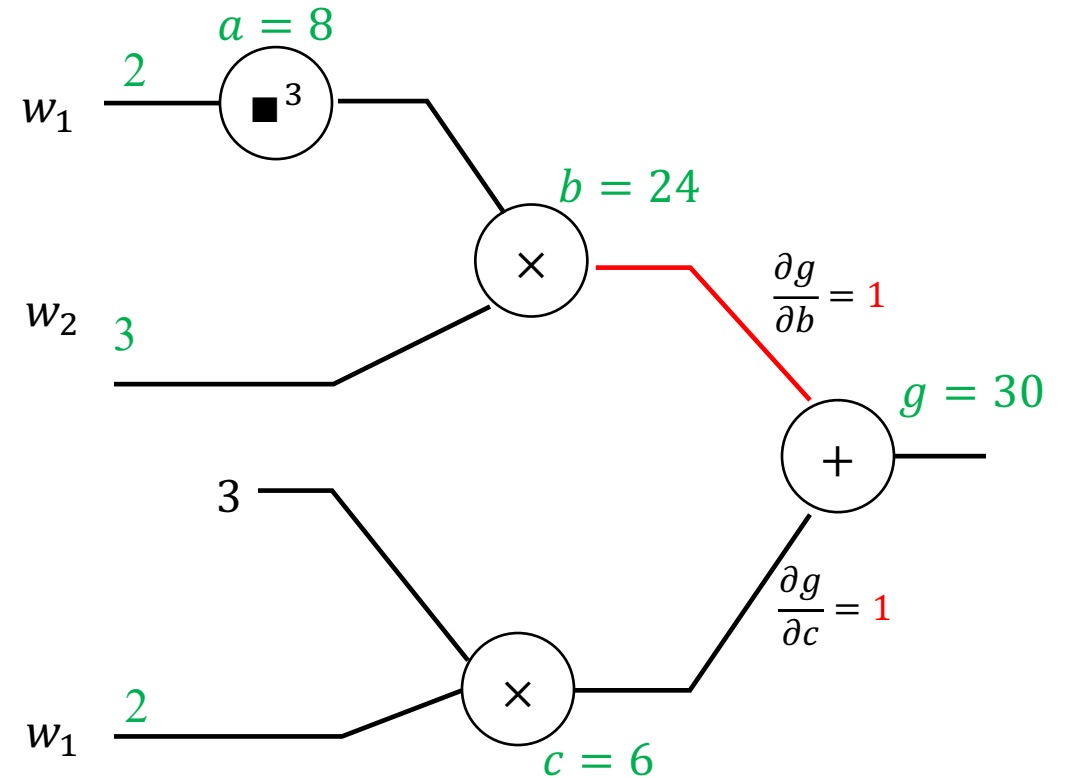
- **Forward step** – substitute your input and get values for each node in the computational graph

- Calculate the gradient backward by **backpropagation**



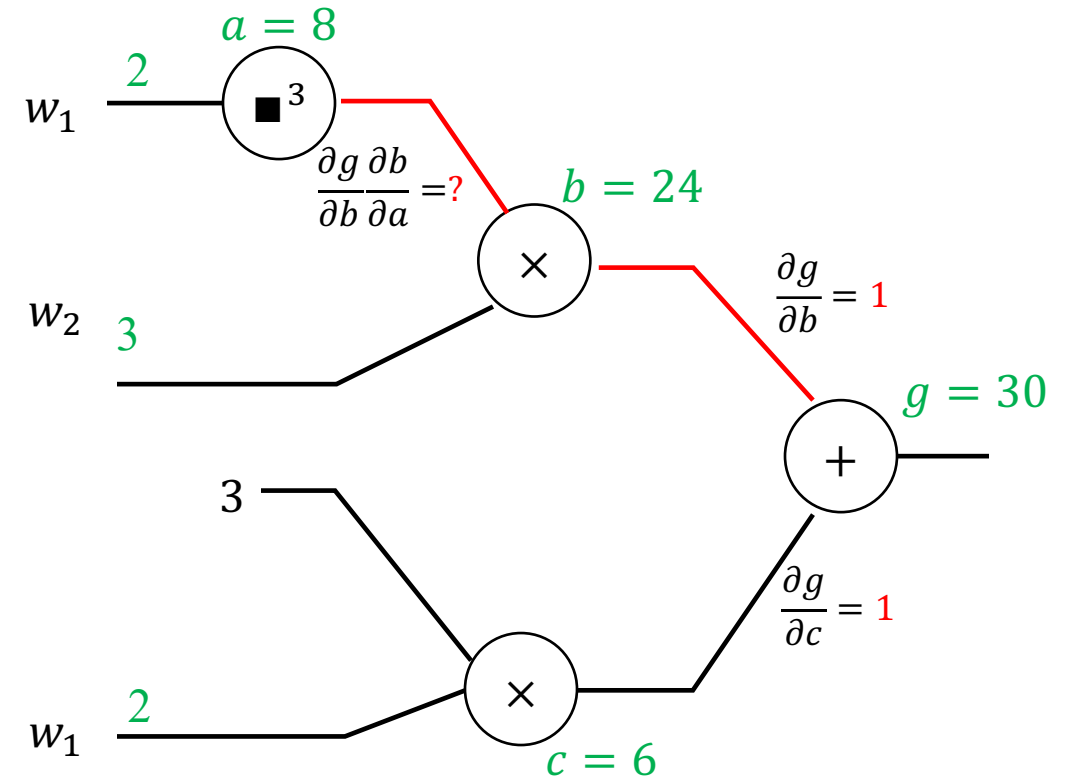
Back Propagation: $g(\mathbf{w}) = w_1^3 w_2 + 3w_1$

- Suppose we have $g(\mathbf{w}) = w_1^3 w_2 + 3w_1$ and want the gradient at $\mathbf{w} = [2, 3]$
- Think of the function as a composition of many functions.
 - Can use derivative chain rule to compute $\partial g / \partial w_1$ and $\partial g / \partial w_2$.
- $g = b + c$
 - $\frac{\partial g}{\partial b} = 1, \frac{\partial g}{\partial c} = 1$



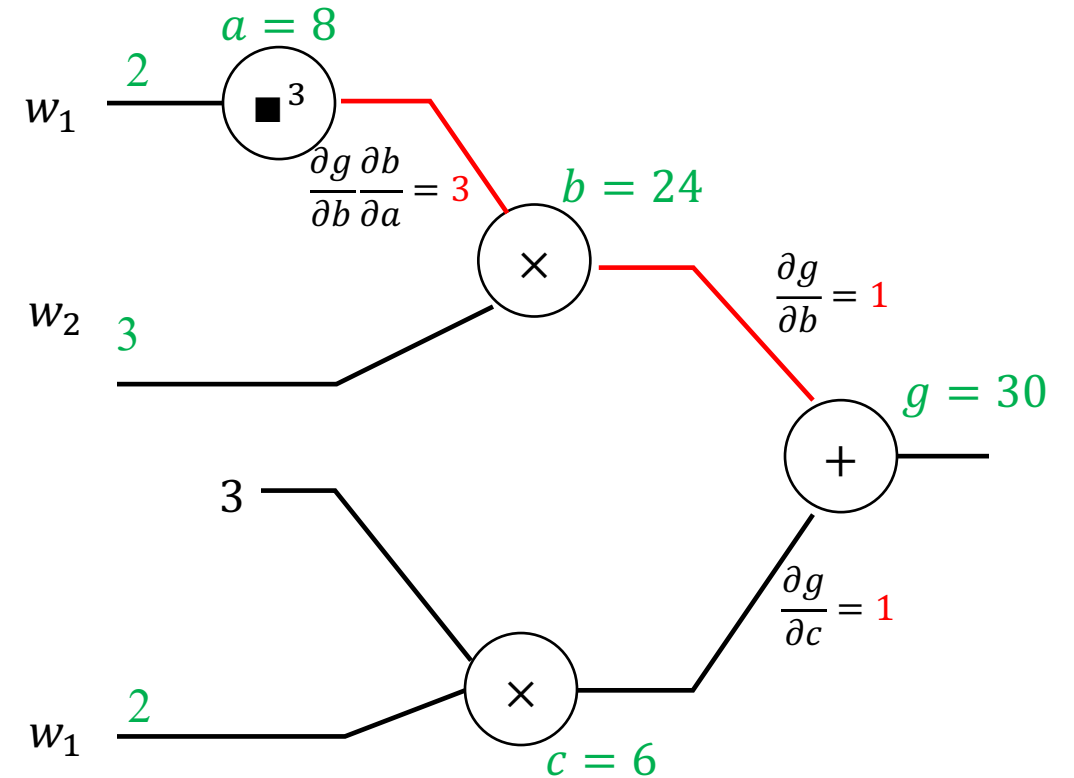
Back Propagation: $g(\mathbf{w}) = w_1^3 w_2 + 3w_1$

- Suppose we have $g(\mathbf{w}) = w_1^3 w_2 + 3w_1$ and want the gradient at $\mathbf{w} = [2, 3]$
- Think of the function as a composition of many functions.
 - Can use derivative chain rule to compute $\partial g / \partial w_1$ and $\partial g / \partial w_2$.
- $g = b + c$
 - $\frac{\partial g}{\partial b} = 1, \frac{\partial g}{\partial c} = 1$
- $b = a \times w_2$
 - $\frac{\partial g}{\partial a} = \frac{\partial g}{\partial b} \frac{\partial b}{\partial a} = \text{?????}$



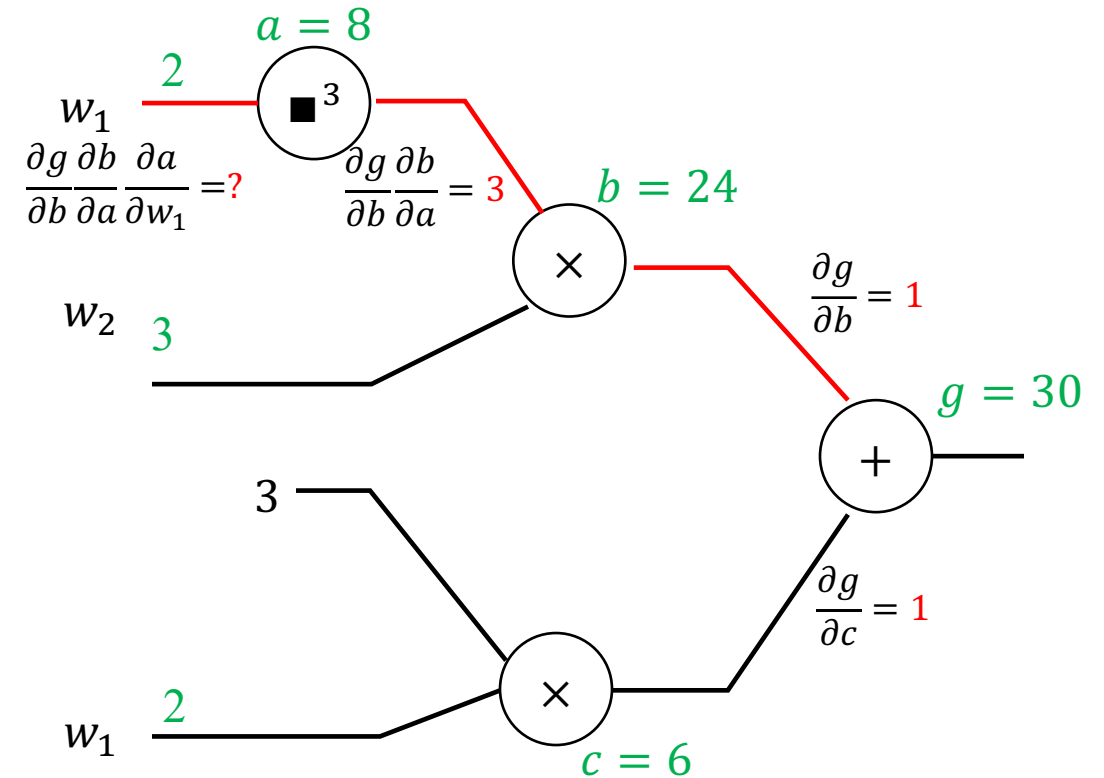
Back Propagation: $g(\mathbf{w}) = w_1^3 w_2 + 3w_1$

- Suppose we have $g(\mathbf{w}) = w_1^3 w_2 + 3w_1$ and want the gradient at $\mathbf{w} = [2, 3]$
- Think of the function as a composition of many functions.
 - Can use derivative chain rule to compute $\partial g / \partial w_1$ and $\partial g / \partial w_2$.
- $g = b + c$
 - $\frac{\partial g}{\partial b} = 1, \frac{\partial g}{\partial c} = 1$
- $b = a \times w_2$
 - $\frac{\partial g}{\partial a} = \frac{\partial g}{\partial b} \frac{\partial b}{\partial a} = 1 \frac{\partial b}{\partial a} = 1 \cdot 3 = 3$



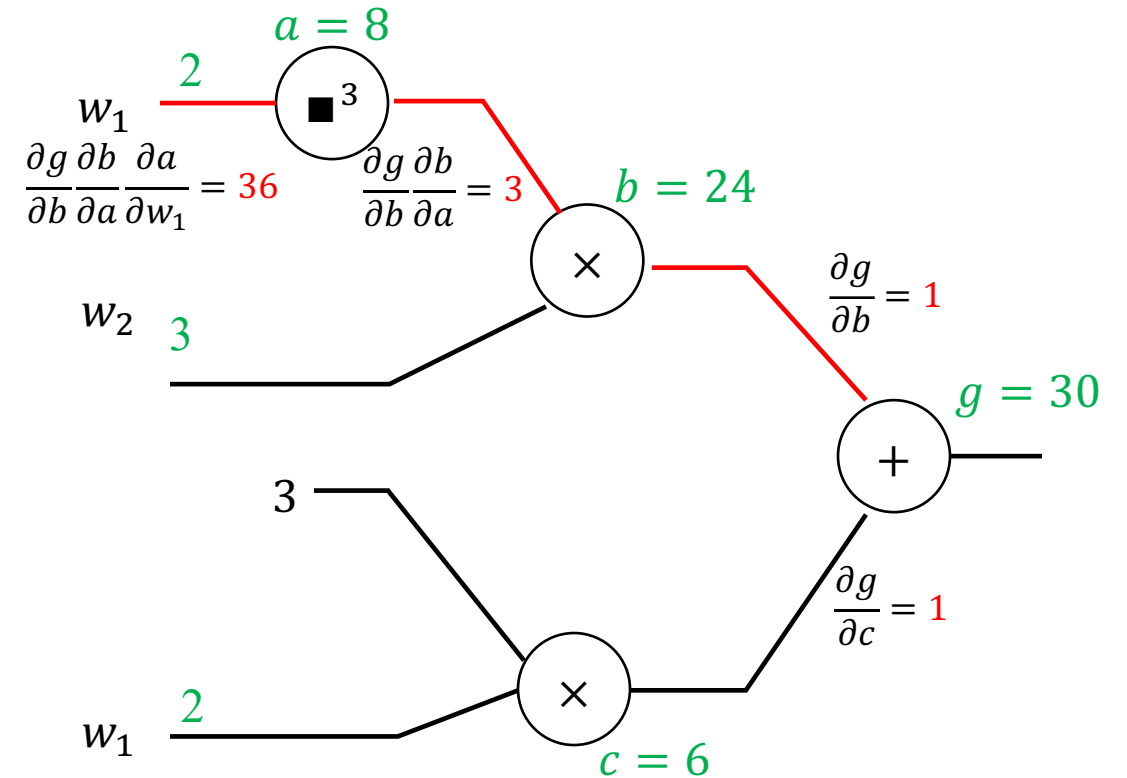
Back Propagation: $g(\mathbf{w}) = w_1^3 w_2 + 3w_1$

- Suppose we have $g(\mathbf{w}) = w_1^3 w_2 + 3w_1$ and want the gradient at $\mathbf{w} = [2, 3]$
- Think of the function as a composition of many functions.
 - Can use derivative chain rule to compute $\partial g / \partial w_1$ and $\partial g / \partial w_2$.
- $g = b + c$
 - $\frac{\partial g}{\partial b} = 1, \frac{\partial g}{\partial c} = 1$
- $b = a \times w_2$
 - $\frac{\partial g}{\partial a} = \frac{\partial g}{\partial b} \frac{\partial b}{\partial a} = 1 \frac{\partial b}{\partial a} = 1 \cdot 3 = 3$
- $a = w_1^3$
 - $\frac{\partial g}{\partial w_1} = \text{?????}$



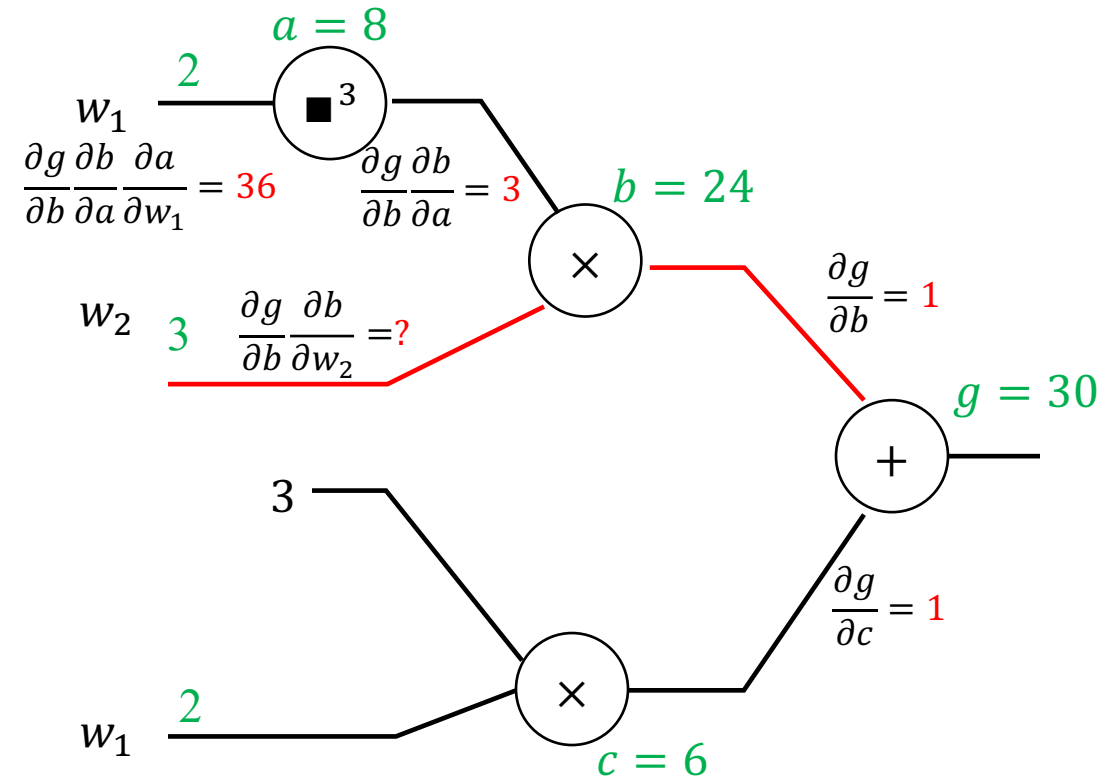
Back Propagation: $g(\mathbf{w}) = w_1^3 w_2 + 3w_1$

- Suppose we have $g(\mathbf{w}) = w_1^3 w_2 + 3w_1$ and want the gradient at $\mathbf{w} = [2, 3]$
- Think of the function as a composition of many functions.
 - Can use derivative chain rule to compute $\partial g / \partial w_1$ and $\partial g / \partial w_2$.
- $g = b + c$
 - $\frac{\partial g}{\partial b} = 1, \frac{\partial g}{\partial c} = 1$
- $b = a \times w_2$
 - $\frac{\partial g}{\partial a} = \frac{\partial g}{\partial b} \frac{\partial b}{\partial a} = 1 \frac{\partial b}{\partial a} = 1 \cdot 3 = 3$
- $a = w_1^3$
 - $\frac{\partial g}{\partial w_1} = \frac{\partial g}{\partial a} \frac{\partial a}{\partial w_1} = 3 \cdot 3w_1^2 = 36$



Back Propagation: $g(\mathbf{w}) = w_1^3 w_2 + 3w_1$

- Suppose we have $g(\mathbf{w}) = w_1^3 w_2 + 3w_1$ and want the gradient at $\mathbf{w} = [2, 3]$
- Think of the function as a composition of many functions.
 - Can use derivative chain rule to compute $\partial g / \partial w_1$ and $\partial g / \partial w_2$.
- $g = b + c$
 - $\frac{\partial g}{\partial b} = 1, \frac{\partial g}{\partial c} = 1$
- $b = a \times w_2$
 - $\frac{\partial g}{\partial a} = \frac{\partial g}{\partial b} \frac{\partial b}{\partial a} = 1 \frac{\partial b}{\partial a} = 1 \cdot 3 = 3$
- $a = w_1^3$
 - $\frac{\partial g}{\partial w_1} = \frac{\partial g}{\partial a} \frac{\partial a}{\partial w_1} = 3 \cdot 3w_1^2 = 36$
- $\frac{\partial g}{\partial w_2} = ???$ Hint: $b = a \times w_2$ may be useful.



Back Propagation: $g(\mathbf{w}) = w_1^3 w_2 + 3w_1$

- Suppose we have $g(\mathbf{w}) = w_1^3 w_2 + 3w_1$ and want the gradient at $\mathbf{w} = [2, 3]$
- Think of the function as a composition of many functions.
 - Can use derivative chain rule to compute $\partial g / \partial w_1$ and $\partial g / \partial w_2$.

- $g = b + c$

- $\frac{\partial g}{\partial b} = 1, \frac{\partial g}{\partial c} = 1$

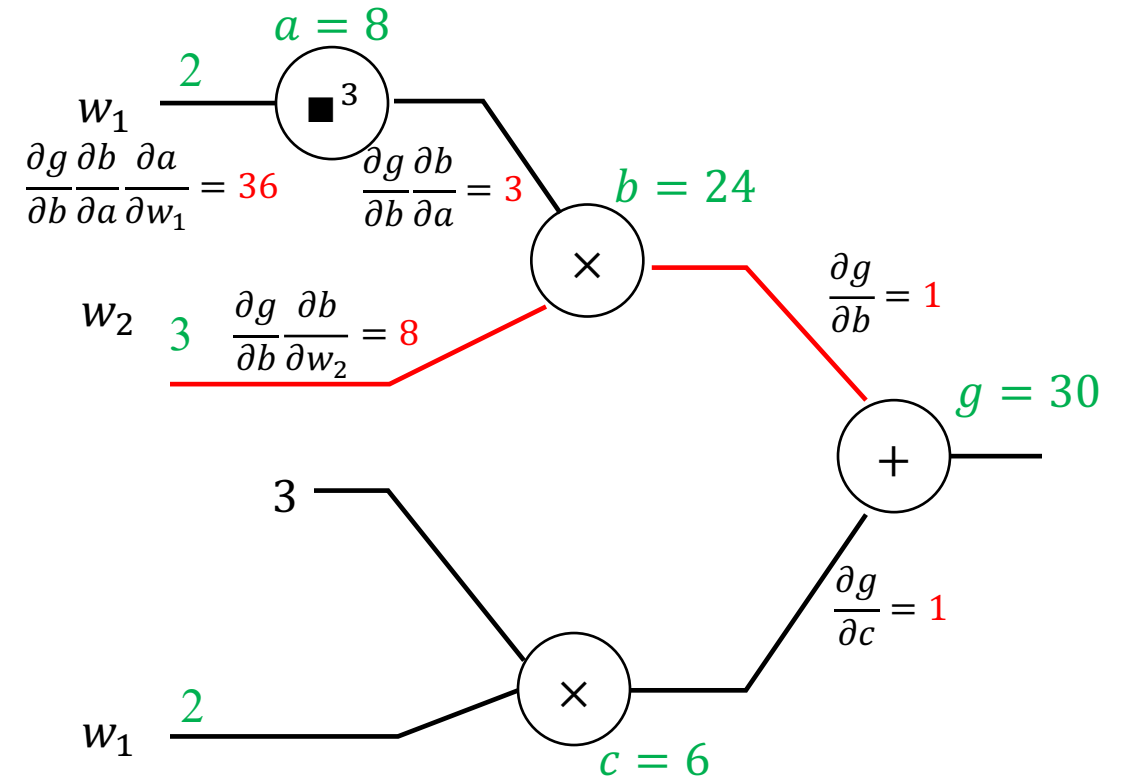
- $b = a \times w_2$

- $\frac{\partial g}{\partial a} = \frac{\partial g}{\partial b} \frac{\partial b}{\partial a} = 1 \frac{\partial b}{\partial a} = 1 \cdot 3 = 3$

- $\frac{\partial g}{\partial w_2} = \frac{\partial g}{\partial b} \frac{\partial b}{\partial w_2} = 1 \frac{\partial b}{\partial w_2} = 1 \cdot 8 = 8$

- $a = w_1^3$

- $\frac{\partial g}{\partial w_1} = \frac{\partial g}{\partial a} \frac{\partial a}{\partial w_1} = 3 \cdot 3w_1^2 = 36$



Back Propagation: $g(\mathbf{w}) = w_1^3 w_2 + 3w_1$

- Suppose we have $g(\mathbf{w}) = w_1^3 w_2 + 3w_1$ and want the gradient at $\mathbf{w} = [2, 3]$
- Think of the function as a composition of many functions.

- $g = b + c$

- $\frac{\partial g}{\partial b} = 1, \frac{\partial g}{\partial c} = 1$

- $b = a \times w_2$

- $\frac{\partial g}{\partial a} = \frac{\partial g}{\partial b} \frac{\partial b}{\partial a} = 1 \frac{\partial b}{\partial a} = 1 \cdot 3 = 3$

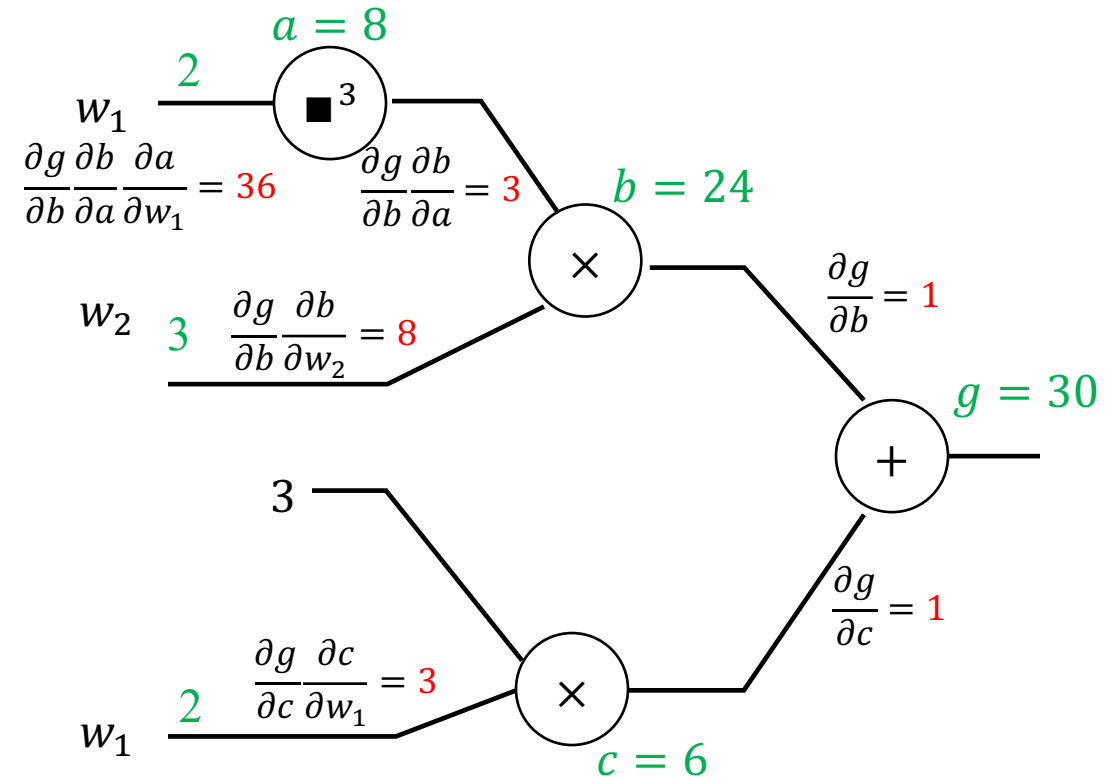
- $\frac{\partial g}{\partial w_2} = \frac{\partial g}{\partial b} \frac{\partial b}{\partial w_2} = 1 \frac{\partial b}{\partial w_2} = 1 \cdot 8 = 8$

- $a = w_1^3$

- $\frac{\partial g}{\partial w_1} = \frac{\partial g}{\partial a} \frac{\partial a}{\partial w_1} = 3 \cdot 3w_1^2 = 36$

- $c = 3w_1$

- $\frac{\partial g}{\partial w_1} = \frac{\partial g}{\partial c} \frac{\partial c}{\partial w_1} = 1 \cdot 3 = 3$



Back Propagation: $g(\mathbf{w}) = w_1^3 w_2 + 3w_1$

- Suppose we have $g(\mathbf{w}) = w_1^3 w_2 + 3w_1$ and want the gradient at $\mathbf{w} = [2, 3]$
- Think of the function as a composition of many functions.

- $g = b + c$

- $\frac{\partial g}{\partial b} = 1, \frac{\partial g}{\partial c} = 1$

- $b = a \times w_2$

- $\frac{\partial g}{\partial a} = \frac{\partial g}{\partial b} \frac{\partial b}{\partial a} = 1 \frac{\partial b}{\partial a} = 1 \cdot 3 = 3$

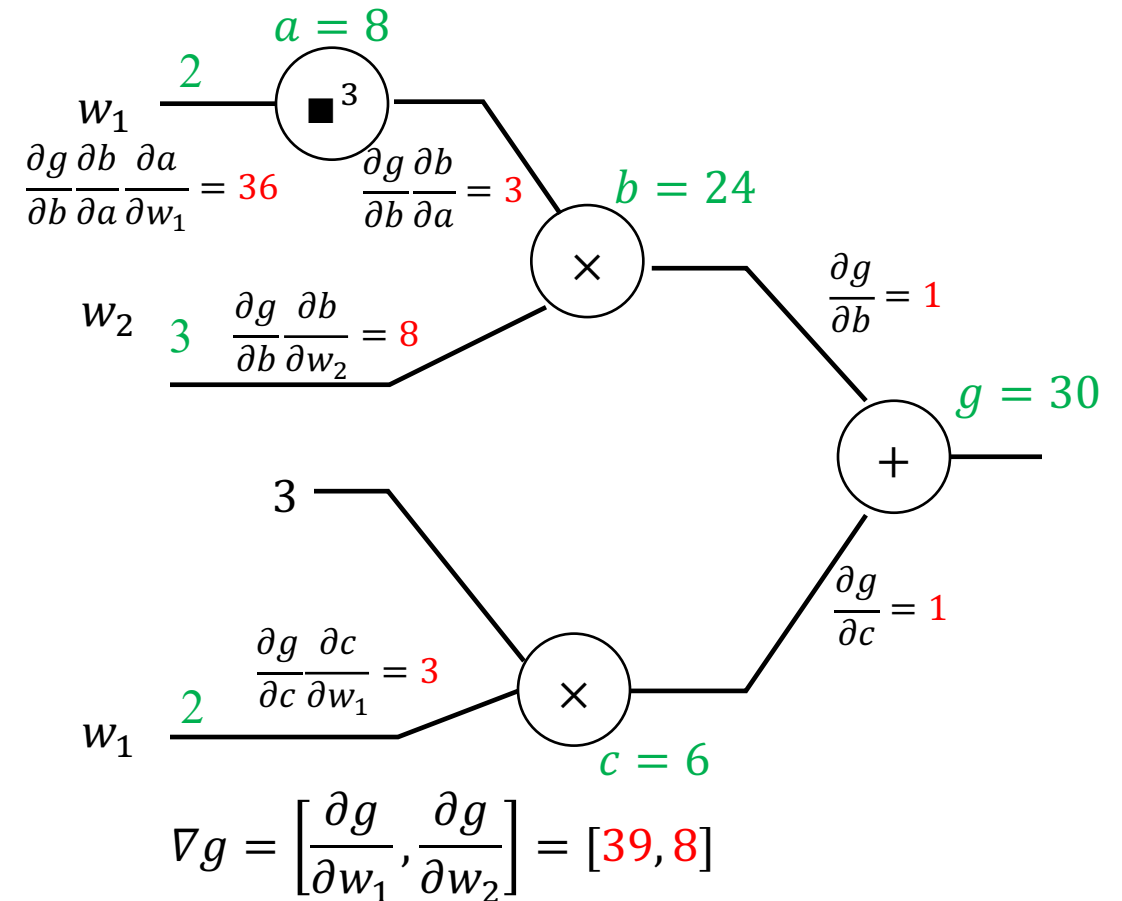
- $\frac{\partial g}{\partial w_2} = \frac{\partial g}{\partial b} \frac{\partial b}{\partial w_2} = 1 \frac{\partial b}{\partial w_2} = 1 \cdot 8 = 8$

- $a = w_1^3$

- $\frac{\partial g}{\partial w_1} = \frac{\partial g}{\partial a} \frac{\partial a}{\partial w_1} = 3 \cdot 3w_1^2 = 36$

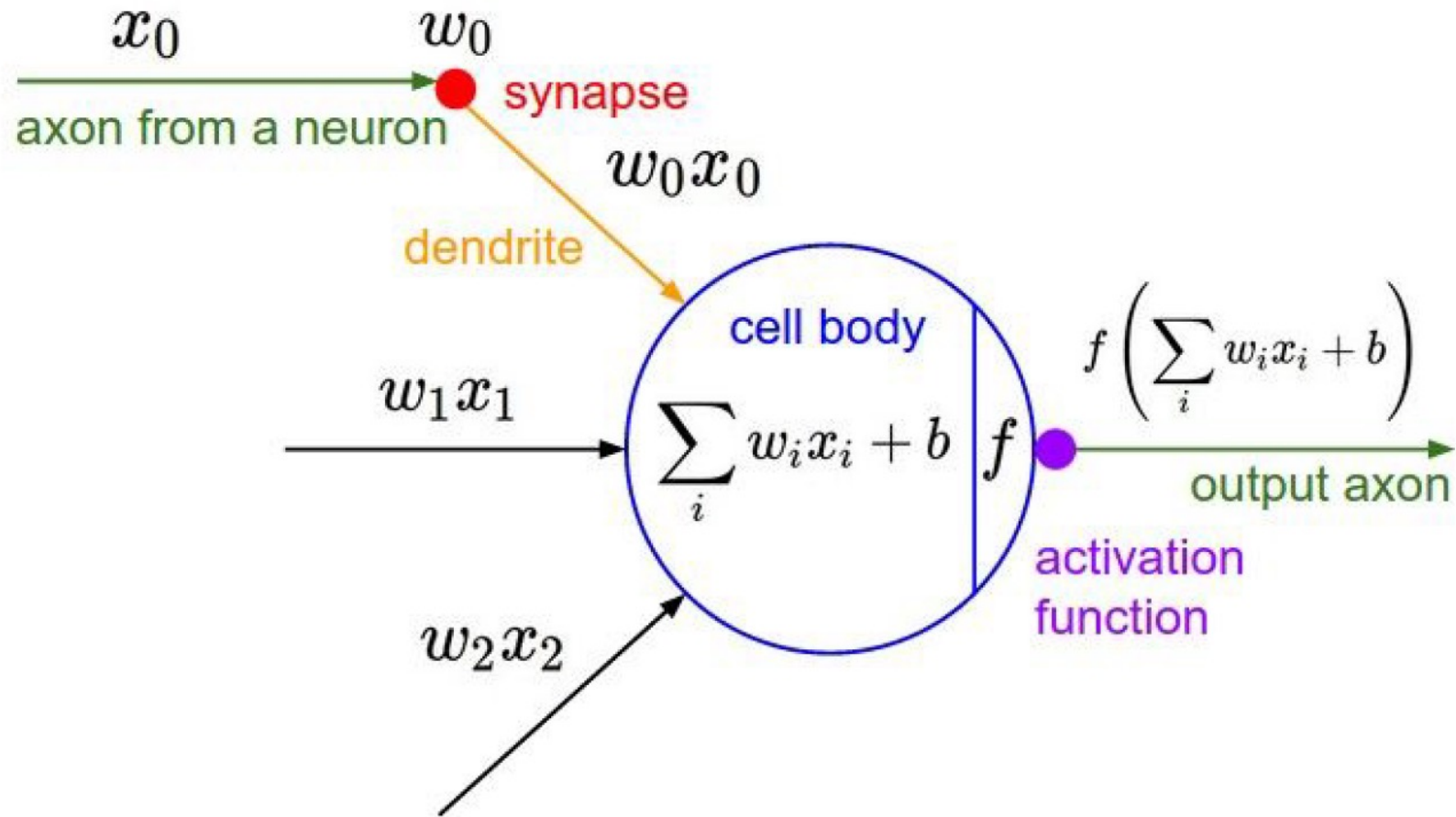
- $c = 3w_1$

- $\frac{\partial g}{\partial w_1} = \frac{\partial g}{\partial c} \frac{\partial c}{\partial w_1} = 1 \cdot 3 = 3$



Activation Functions

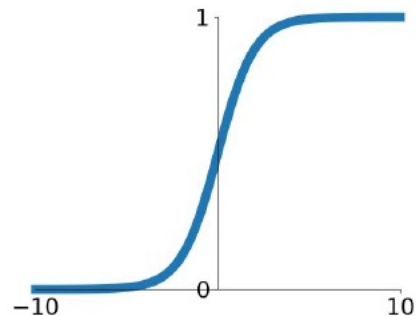
Activation Functions



Activation Functions

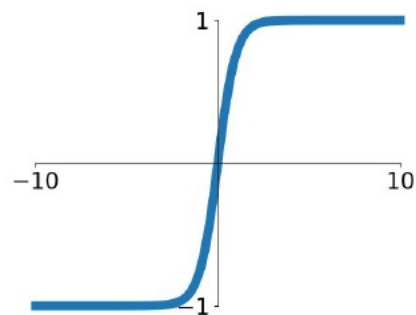
Sigmoid

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$



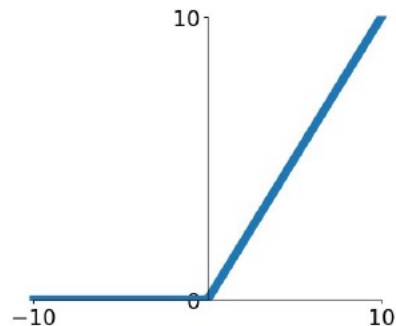
Tanh

$$\tanh(x)$$



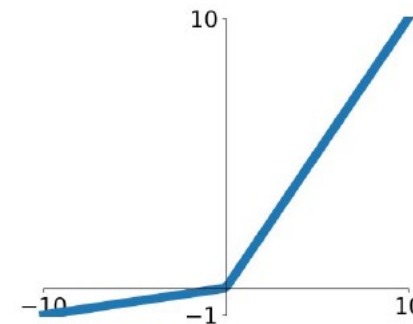
ReLU

$$\max(0, x)$$



Leaky ReLU

$$\max(0.1x, x)$$



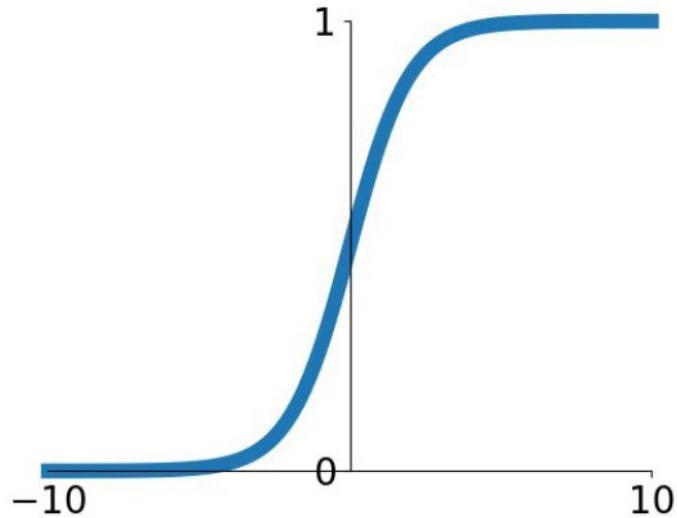
ELU

$$\begin{cases} x & x \geq 0 \\ \alpha(e^x - 1) & x < 0 \end{cases}$$

Reference:

[Activation Functions — ML Glossary documentation \(ml-cheatsheet.readthedocs.io\)](https://ml-cheatsheet.readthedocs.io/)

Activation Functions: Sigmoid



Sigmoid

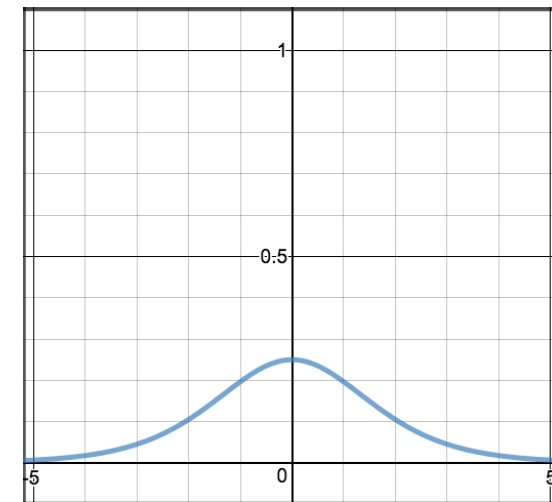
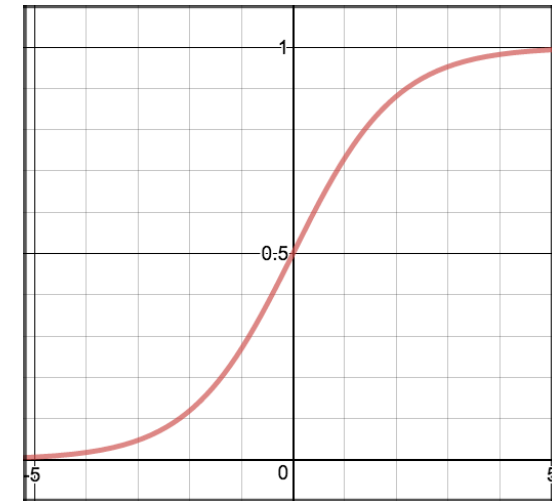
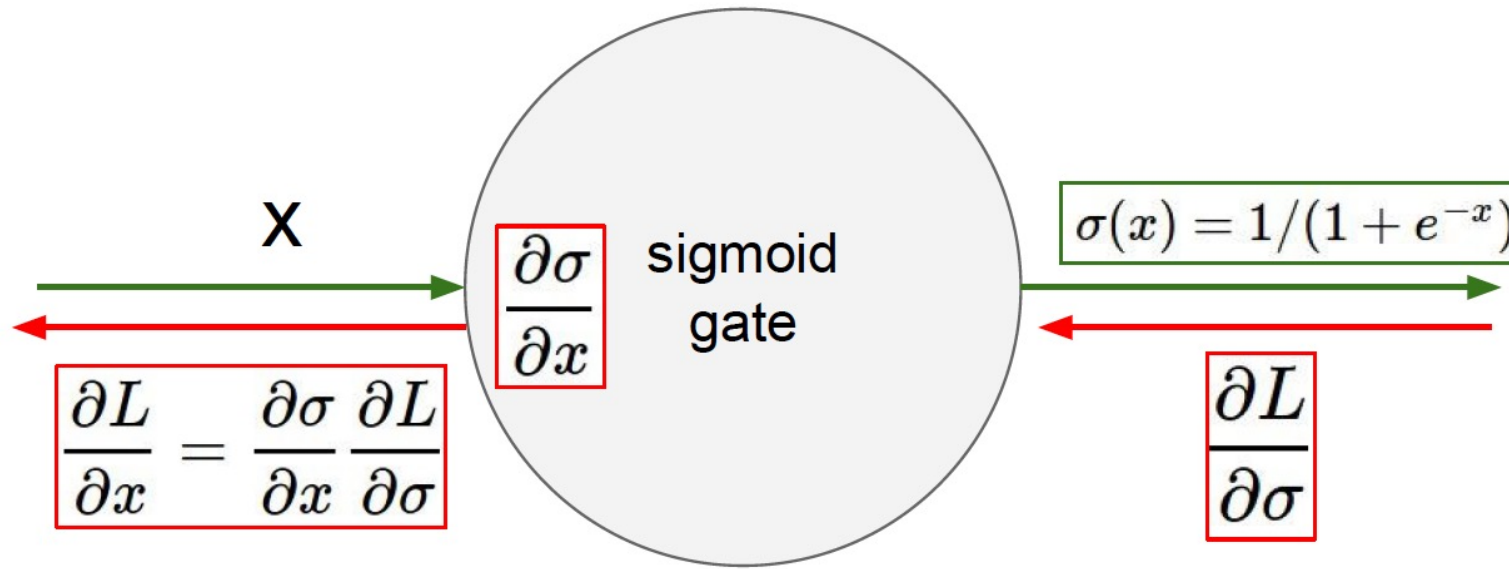
$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

- Squashes numbers to range $[0,1]$
- Historically popular since they have nice interpretation as a saturating “firing rate” of a neuron

Three problems:

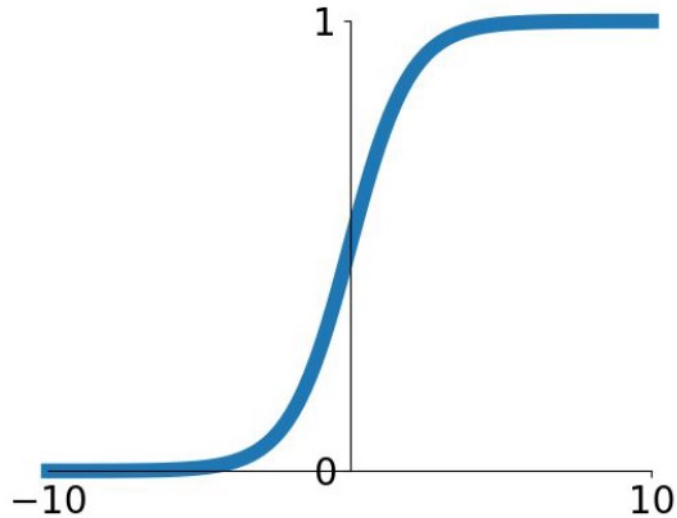
1. Saturated neurons “kill” the gradients

Activation Functions: Sigmoid



- What happens when $x < -5$?
- What happens when $x = 0$?
- What happens when $x > 5$?

Activation Functions: Sigmoid



Sigmoid

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

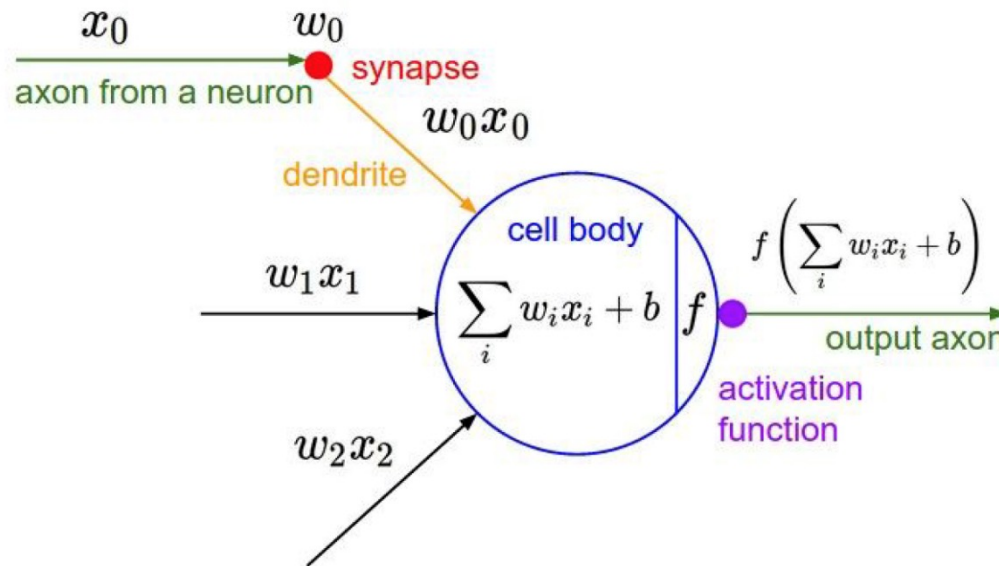
- Squashes numbers to range $[0,1]$
- Historically popular since they have nice interpretation as a saturating “firing rate” of a neuron

Three problems:

1. Saturated neurons “kill” the gradients
2. Sigmoid outputs are not zero-centered

Activation Functions: Sigmoid

- Consider what happens when the input to a neuron (x) is always positive:



$$f\left(\sum_i w_i x_i + b\right)$$

What can we say about the gradients on $\mathbf{w}$?

Activation Functions: Sigmoid

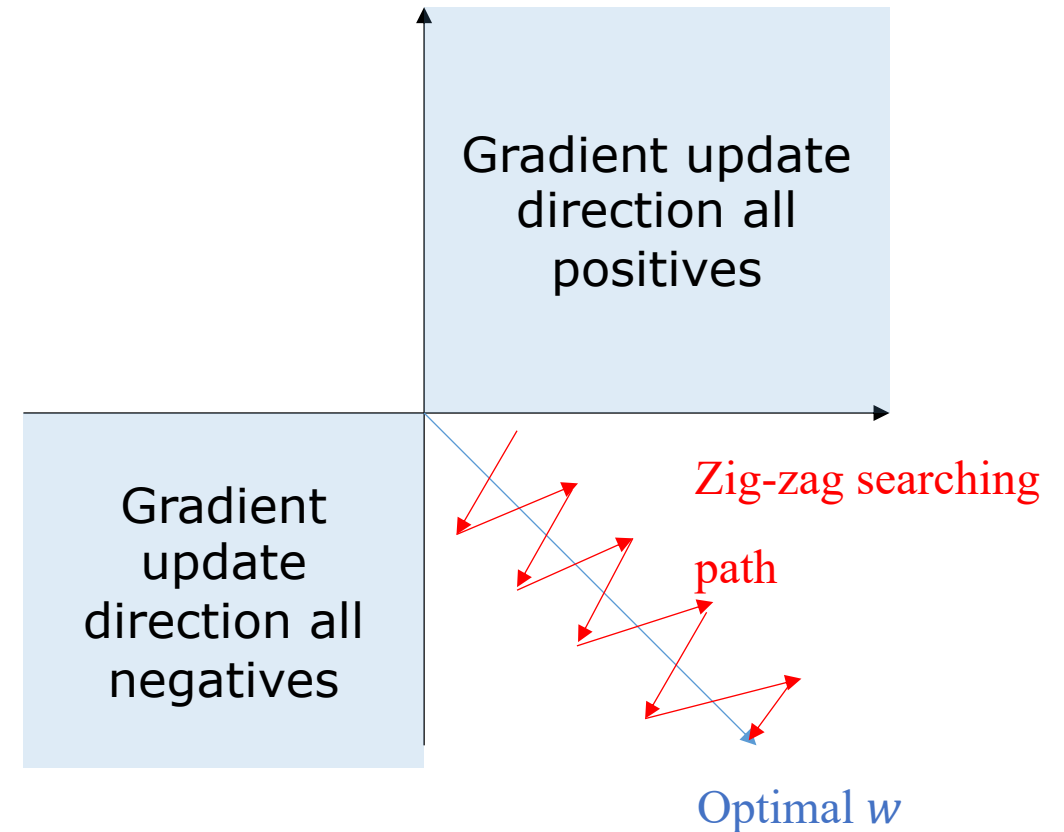
- Consider what happens when the input to a neuron (x) is always positive:

$$f\left(\sum_i w_i x_i + b\right)$$

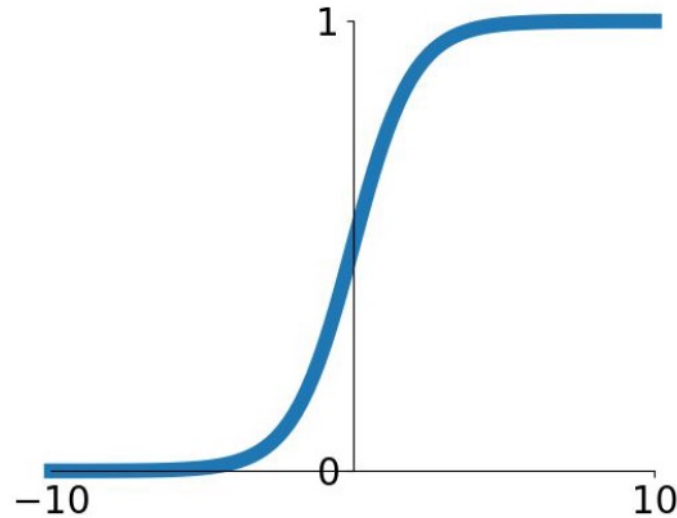
What can we say about the gradients on $\mathbf{w}$?

Always all positive or all negative

A zero-centered activation is preferred to avoid the zig-zag path



Activation Functions: Sigmoid



Sigmoid

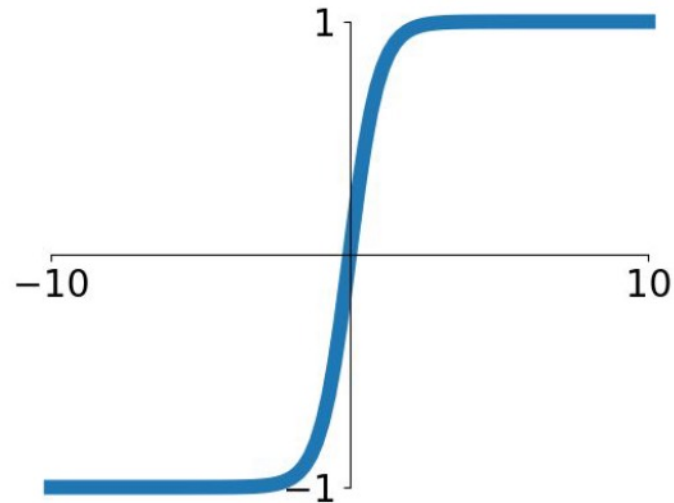
$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

- Squashes numbers to range $[0,1]$
- Historically popular since they have nice interpretation as a saturating “firing rate” of a neuron

Three problems:

1. Saturated neurons “kill” the gradients
2. Sigmoid outputs are not zero-centered
3. $\exp()$ is a bit computationally expensive

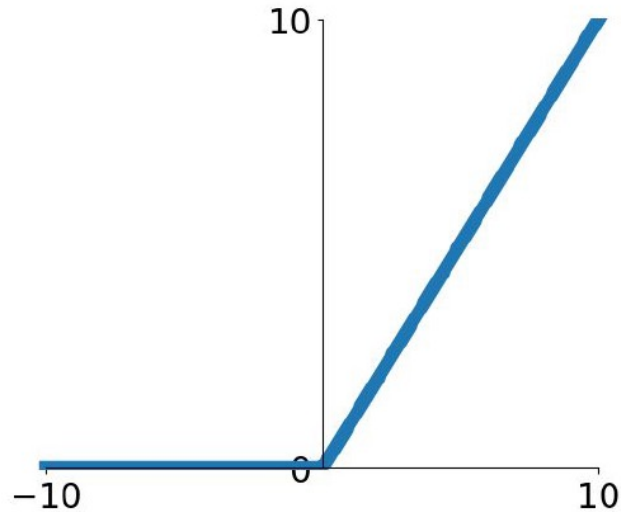
Activation Functions: Tanh



Tanh

- Squashes numbers to range $[-1,1]$
- zero centered (nice)
- still kills gradients when saturated :(

Activation Functions: ReLU

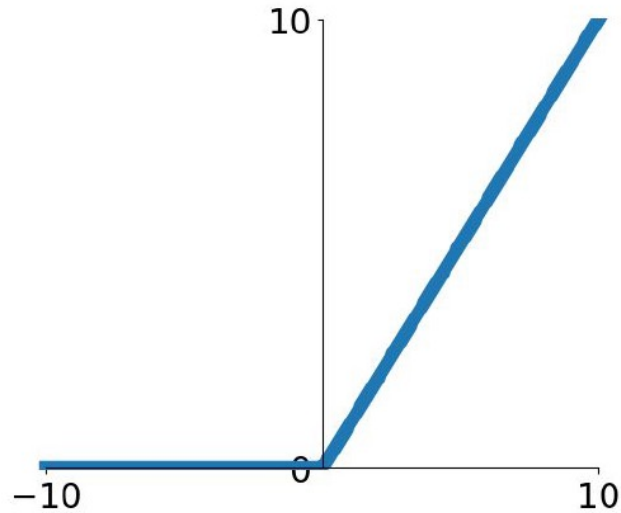


ReLU
(Rectified Linear Unit)

Computes $f(x) = \max(0, x)$

- Does not saturate (in +region)
- Very computationally efficient
- Converges much faster than sigmoid/tanh in practice (e.g. 6x)

Activation Functions: ReLU



ReLU

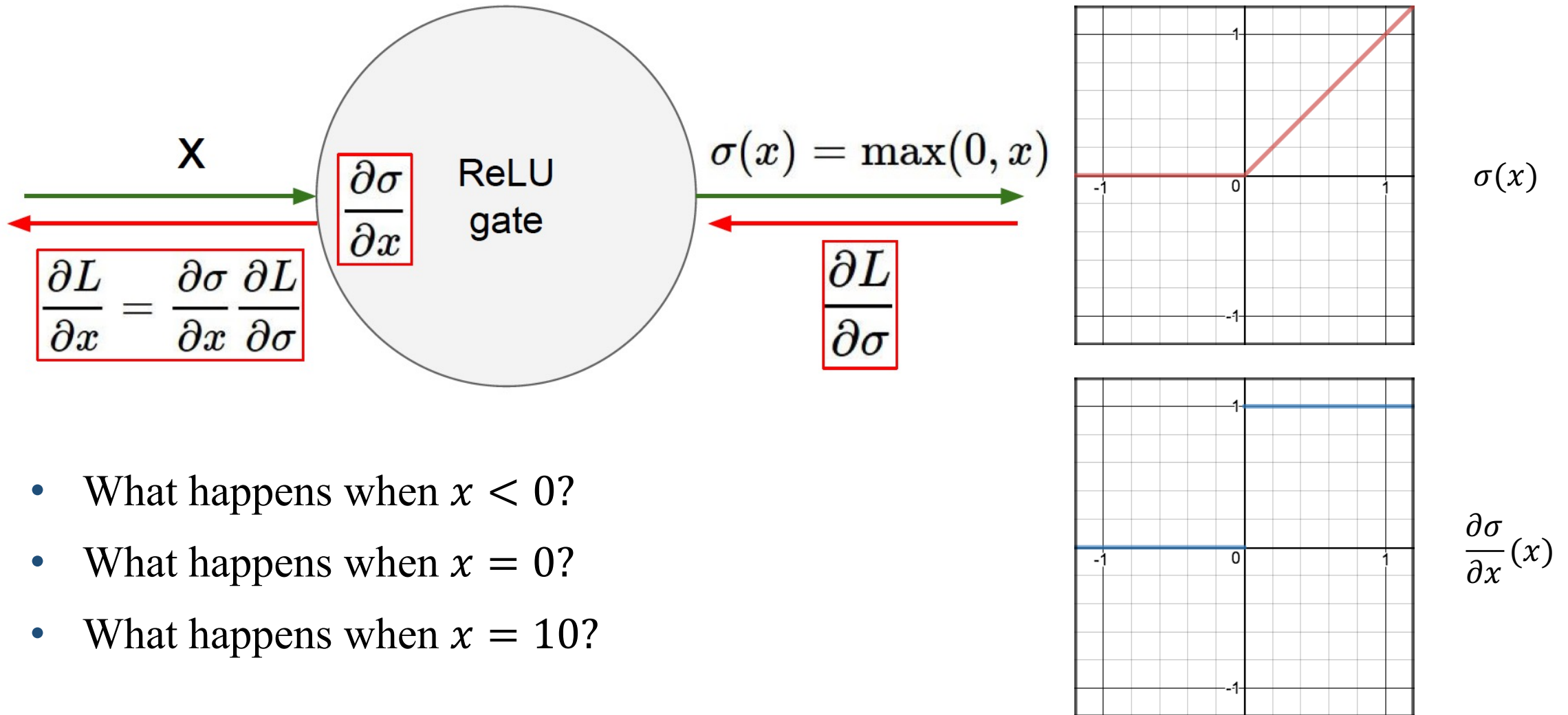
(Rectified Linear Unit)

Computes $f(x) = \max(0, x)$

- Does not saturate (in +region)
- Very computationally efficient
- Converges much faster than sigmoid/tanh in practice (e.g. 6x)
- **Not zero-centered output**

What is the gradient when $x < 0$?

Activation Functions: ReLU

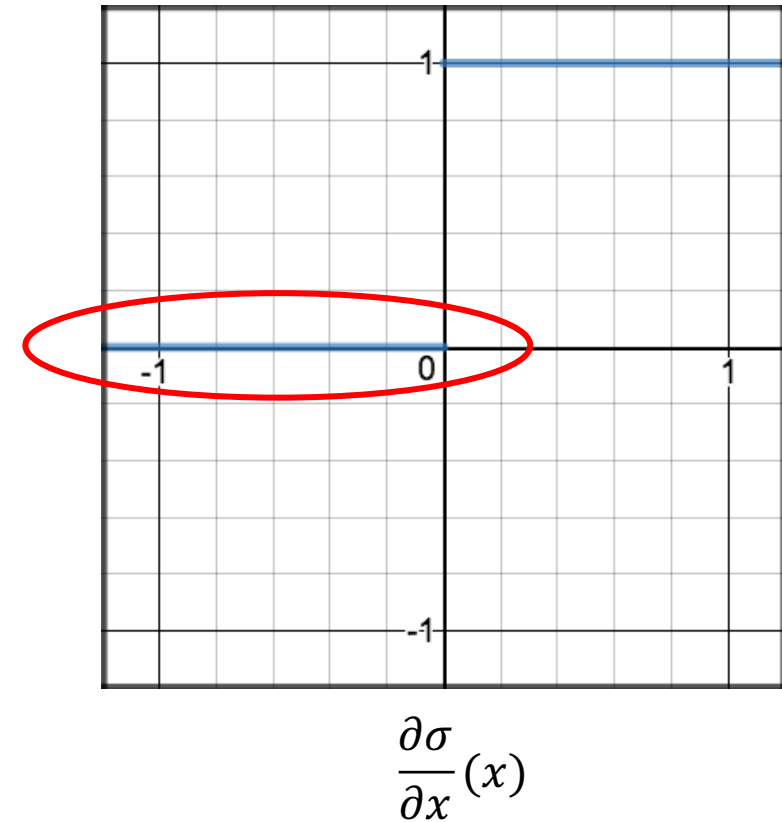


- What happens when $x < 0$?
- What happens when $x = 0$?
- What happens when $x = 10$?

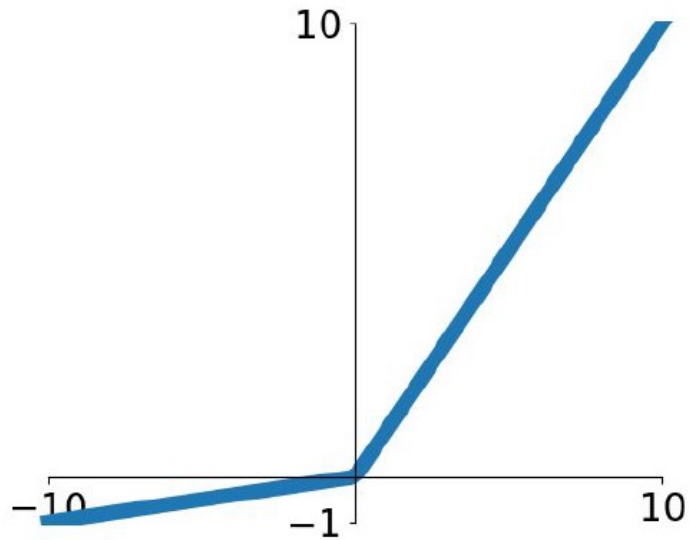
Activation Functions: ReLU

- The dying ReLU Problem:
 - When the input to ReLU is negative, the gradient will be **zero**!
 - Zero gradient will not produce any updates
 - Once you get stuck at zero gradient, you will never get your weights updated again!

⇒ people like to initialize ReLU neurons with slightly positive intercept w_0 (e.g. 0.01)



Activation Functions: Leaky ReLU



- Does not saturate
- Computationally efficient
- Converges much faster than sigmoid/tanh in practice! (e.g. 6x)
- **will not “die”.**

Leaky ReLU

$$f(x) = \max(\alpha x, x)$$

← **backprop into α**

By default $\alpha = 0.01$

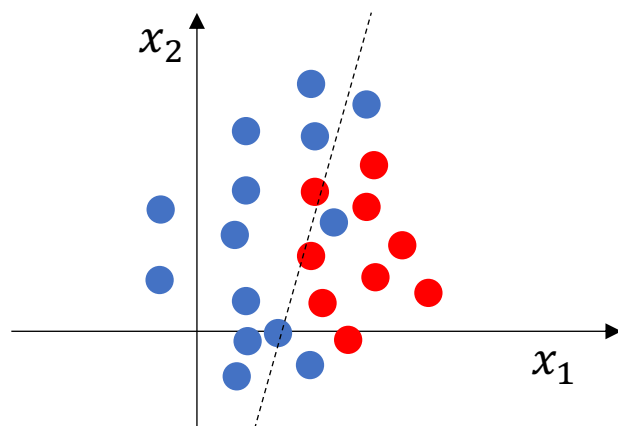
[Mass et al., 2013]

[He et al., 2015]

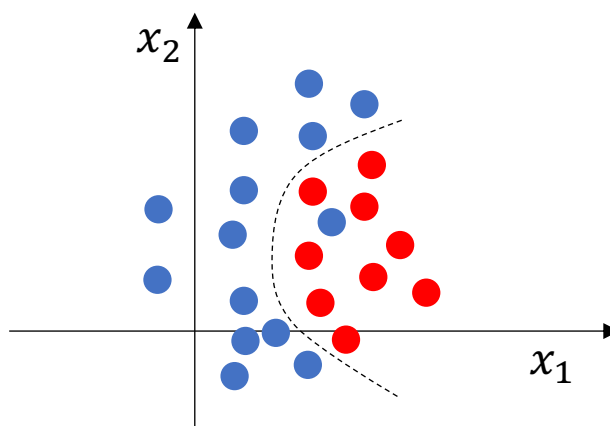
Regularization

Overfitting

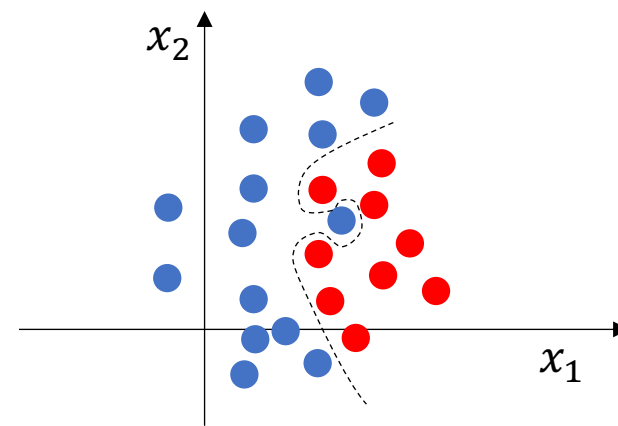
- **Overfitting:** The model fits the training data well but doesn't work well on unknown test data
- Fundamental trade-off: A more complex model may have higher potential to overfit



High bias



Good trade-off



High variance

- Neural networks are easy to overfit because of its model complexity and non-linearity

Regularization

- Regularization is a technique to mitigate the overfitting problem
- In its simplest form, if we have a **minimization** problem:

$$\min_w J(w) = \min_w \sum_{i=1}^m \mathcal{L}(\hat{y}_i, y_i; w)$$

- $\mathcal{L}(\hat{y}_i, y_i; w)$ is the loss for each instance. For classification, it could be cross-entropy, for regression, it could be sum of square errors.

- Regularization is to add a penalty **$R(w)$** to large values of w

$$\min_w J_R(w) = \min_w \sum_{i=1}^m \mathcal{L}(\hat{y}_i, y_i; w) + \lambda R(w)$$

λ controls the strengths of the penalty

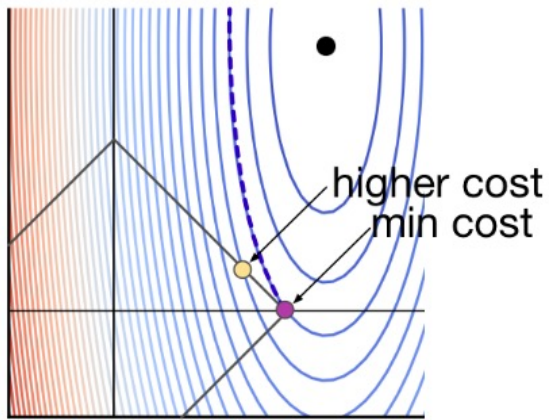
Regularization

- Regularization is to add a penalty $R(w)$ to large values of w

$$\min_w J_R(w) = \min_w \sum_{i=1}^m \mathcal{L}(\hat{y}_i, y_i; w) + \lambda R(w)$$

- L1 regularization:

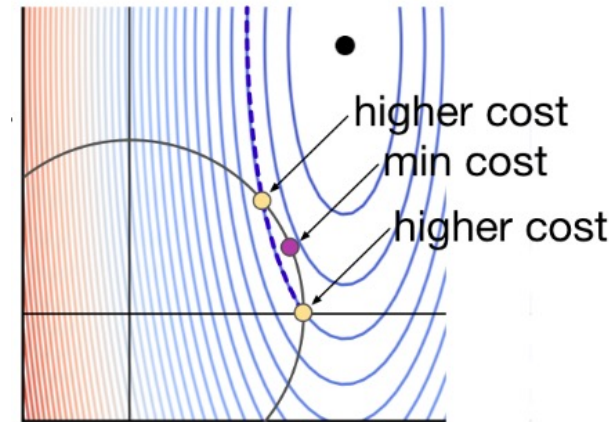
- $R(w) = \sum_j |w_j|$



L1 is often sparser

- L2 regularization:

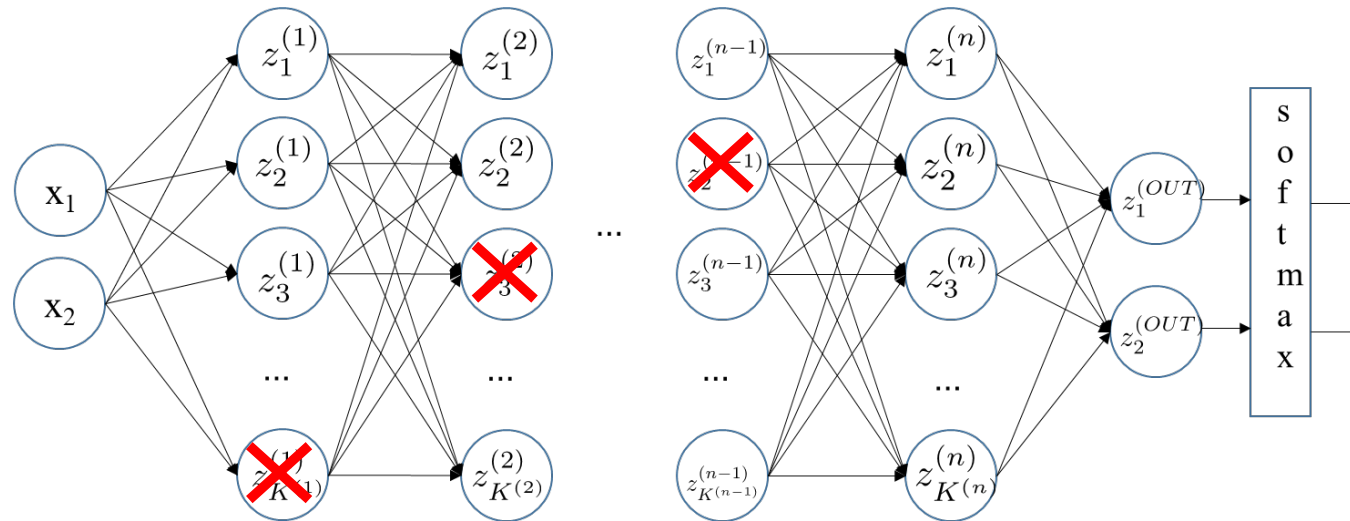
- $R(w) = \|w\|^2 = w^T w = \sum_j w_j^2$



[Image from <https://explained.ai/regularization/L1vsL2.html>]

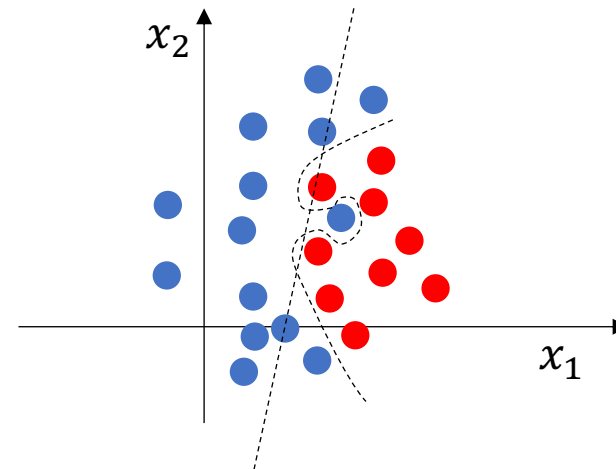
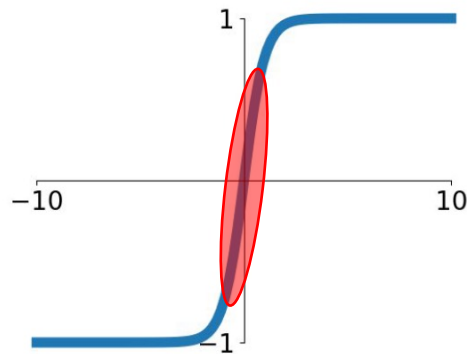
Why Regularization Reduce Overfitting?

- The general idea of regularization is to keep the weights small
- Why does it help?
 - When the weights get close to zero, you can think of some of the neurons are deactivated. This simplifies your model.



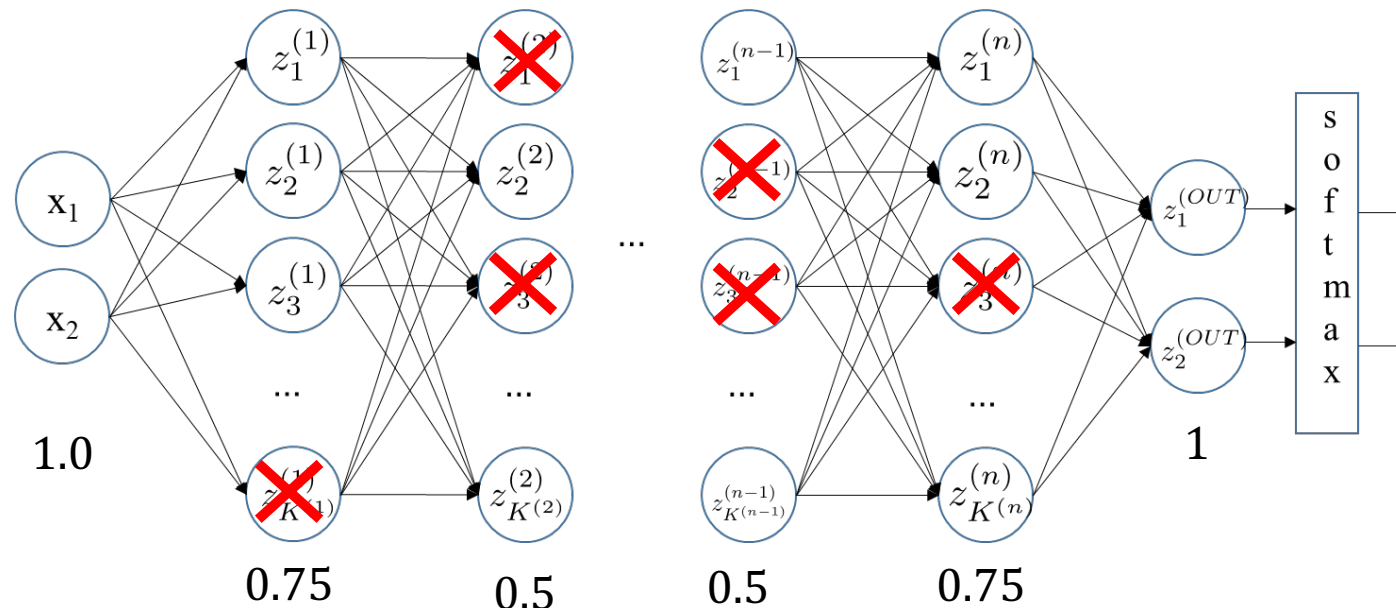
Why Regularization Reduce Overfitting?

- The general idea of regularization is to keep the weights small
- Why does it help?
 - When the weights get close to zero, you can think of some of the neurons are deactivated. This simplifies your model.
 - For some activation function like tanh and sigmoid, small weights around zero makes it closer to a (simple) linear model.



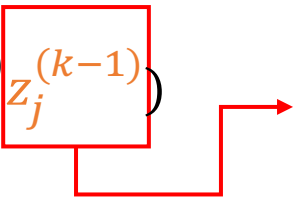
Other Regularization Techniques

- Dropout – not to rely on a few features, spread out the weights
 - In the **training phase**, randomly deactivate some of the nodes in each instance/batch/mini-batch
 - In the **test phase**, we use the whole network without dropout.



Other Regularization Techniques

- Dropout
 - Randomly deactivate some of the nodes in each instance/batch/mini-batch
 - How to implement dropout?
 - Let's say you want to keep 80% of your neurons at a layer
 - For each neuron at the layer
 - Generate a random number in the range $[0,1]$ and compare it with 0.8
 - If the random number is smaller than 0.8 keep the neuron, otherwise, multiply the output of the neuron with 0 (i.e., deactivation).
 - Inverted dropout - divide the output of each neuron at this layer by 0.8. **Why??**

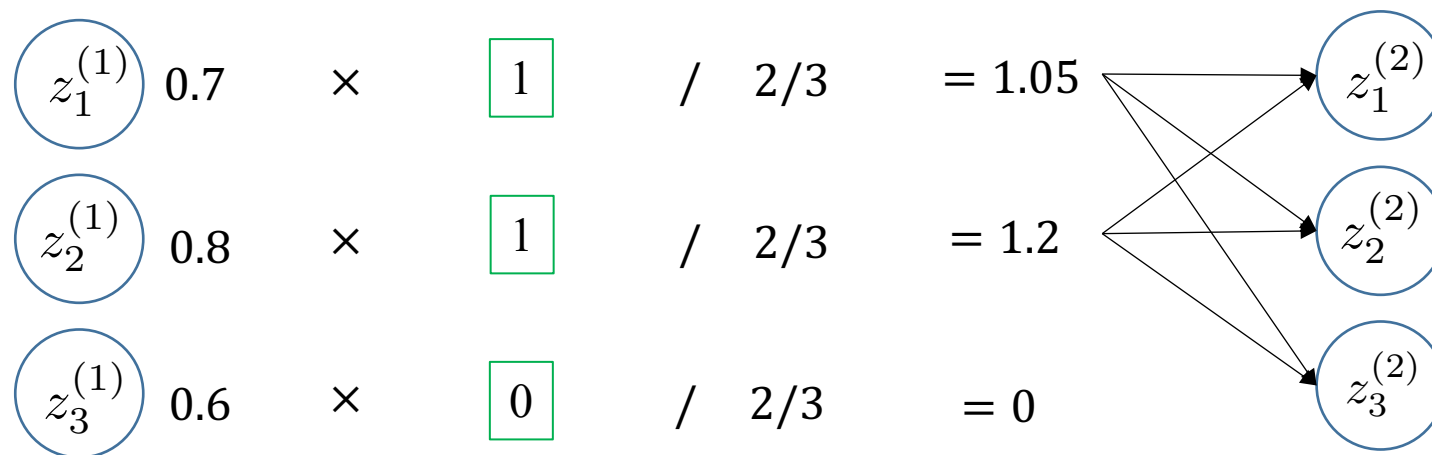
$$z_i^{(k)} = g\left(\sum_j W_{i,j}^{(k-1,k)} z_j^{(k-1)}\right)$$


- Remind that in the test phase, we use the whole network
- The values for these neurons are 20% less than usual. Rescale it back to make sure both training and test phases have similar scale.

Other Regularization Techniques

- Dropout - Example

Keep prob = $2/3$



Keep/drop randomly

Other Regularization Techniques

- Data Augmentation
 - Data transformation - rotation, zoom in/out
 - Add small noise

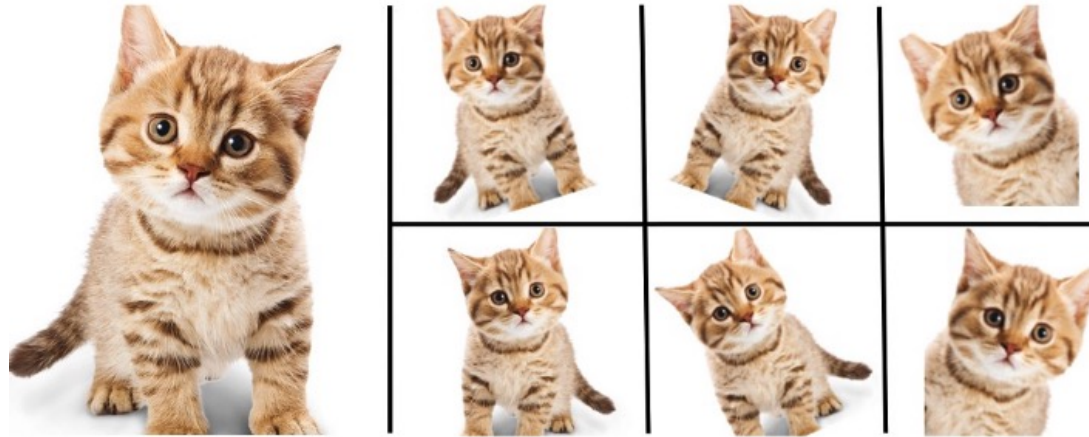
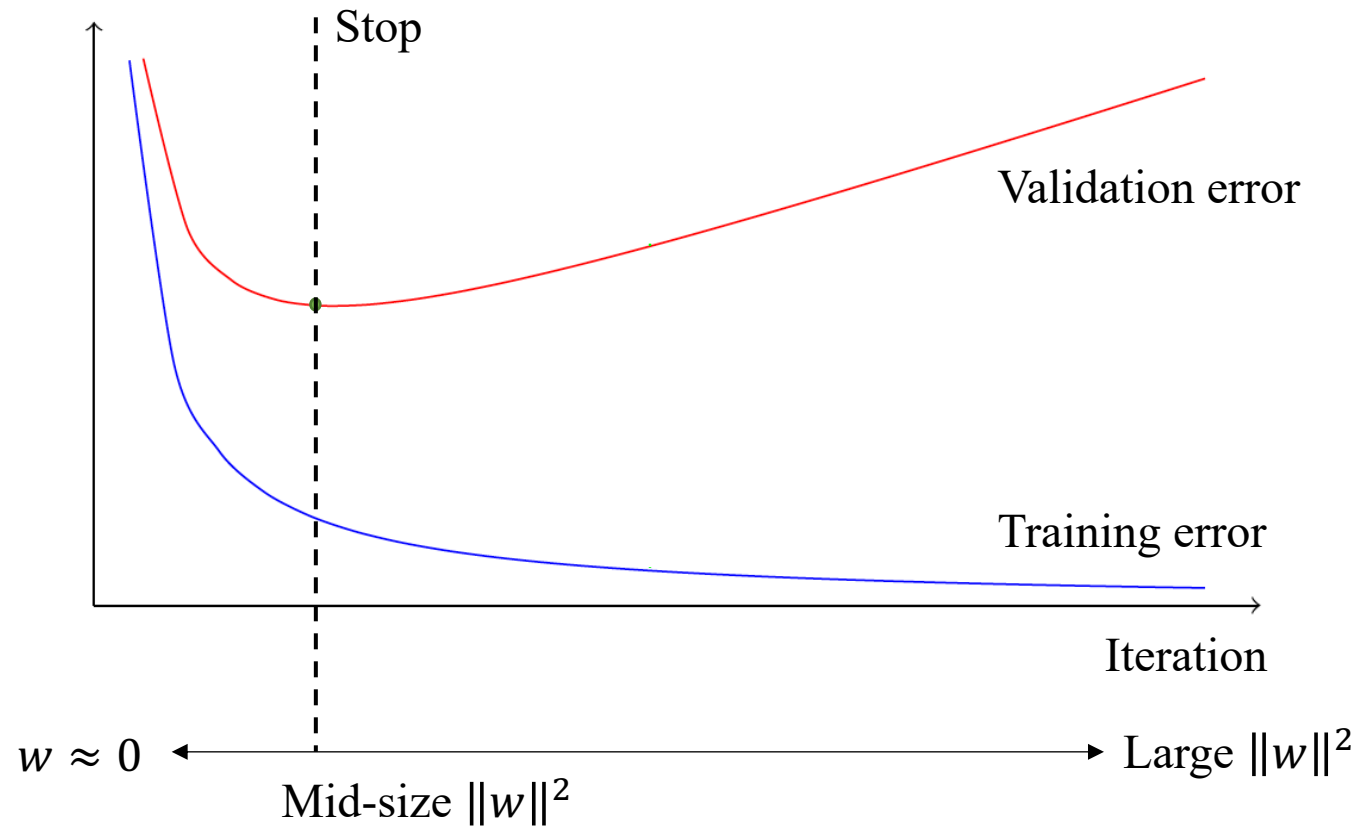


Image source: <https://nanonets.com/blog/data-augmentation-how-to-use-deep-learning-when-you-have-limited-data-part-2/>

Other Regularization Techniques

- Early stopping – Stop when the validation error gets worse



Optimization

Recap: Gradient Descent v.s. Stochastic Gradient Descent



Batch gradient
descent
(min-batch
size= m)

- Heavy calculation per iteration
- High memory requirement

Mini-batch
gradient descent
(min-batch size
not too large)

- Fastest
- Vectorization enables speed up by GPUs
- Can fit in memory

Stochastic
gradient descent
(min-batch
size=1)

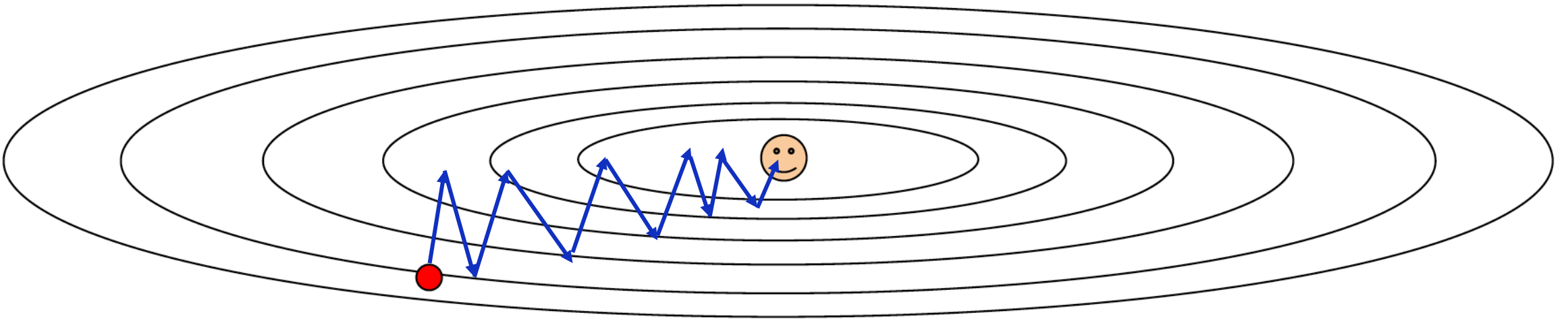
- Loose vectorization speed up
- Large variances of the updates

Choose a mini-batch size in-between 1 and m in practice

Image adapted from: <https://sweta-nit.medium.com/batch-mini-batch-and-stochastic-gradient-descent-e9bc4cacd461>

Problems with Mini-batch SGD

- What if loss changes quickly in one direction and slowly in another?
- What does Mini-batch SGD do?



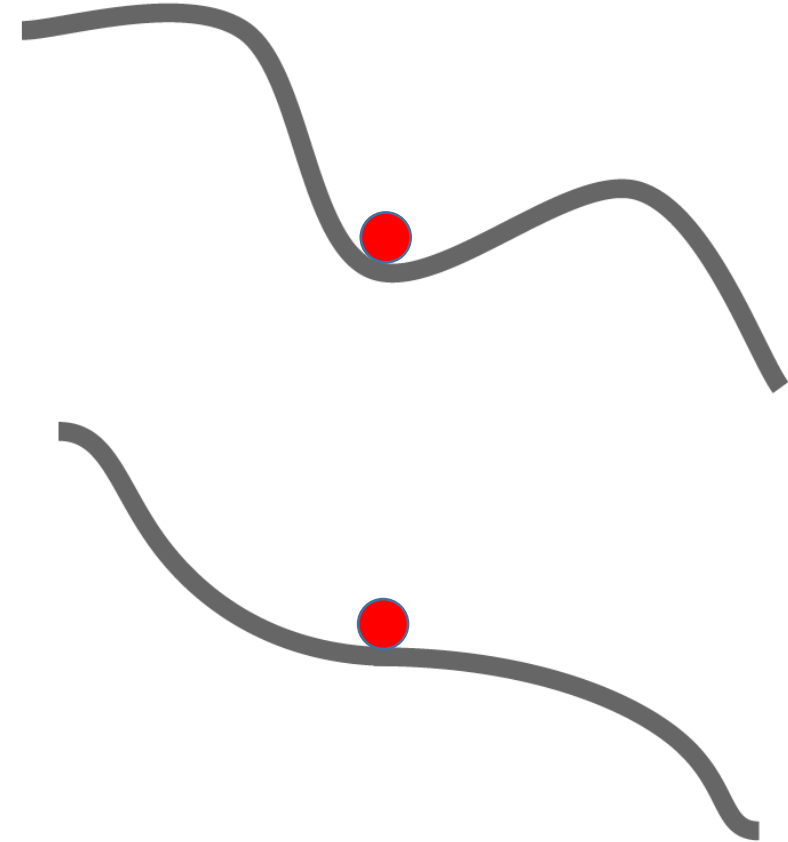
SGD often takes noisy updates

Problems with Mini-batch SGD

What if the loss function has a **local minima** or **saddle point**?

Zero gradient, gradient descent gets stuck

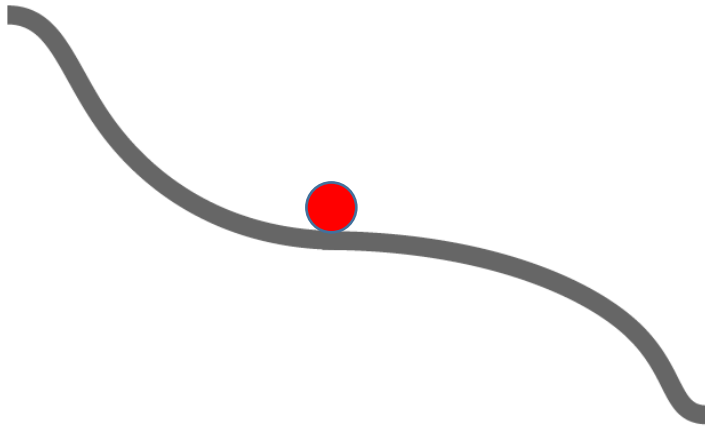
Saddle points much more common in high dimension



Dauphin et al, “Identifying and attacking the saddle point problem in high-dimensional non-convex optimization”, NIPS 2014

Exponentially Weighted Moving Average

- How to address the zero-gradient problem happens at saddle points?
- **Motivation:** If we are moving towards a direction recently, we can keep moving with a speed that is similar to the previous several steps
- **Idea:** Keep track of the velocity in the recent updates



We can use **exponentially weighted moving average** to maintain the recent update velocity

Exponentially Weighted Moving Average

- Definition: $v_t = \rho v_{t-1} + (1 - \rho)\theta_t$
- Example – Temperature

Day t	Temperature θ_t (in $^{\circ}\text{C}$)
1	24
2	29
3	25
4	23
5	...

Let $\rho = 0.5$

$$v_0 = 0$$

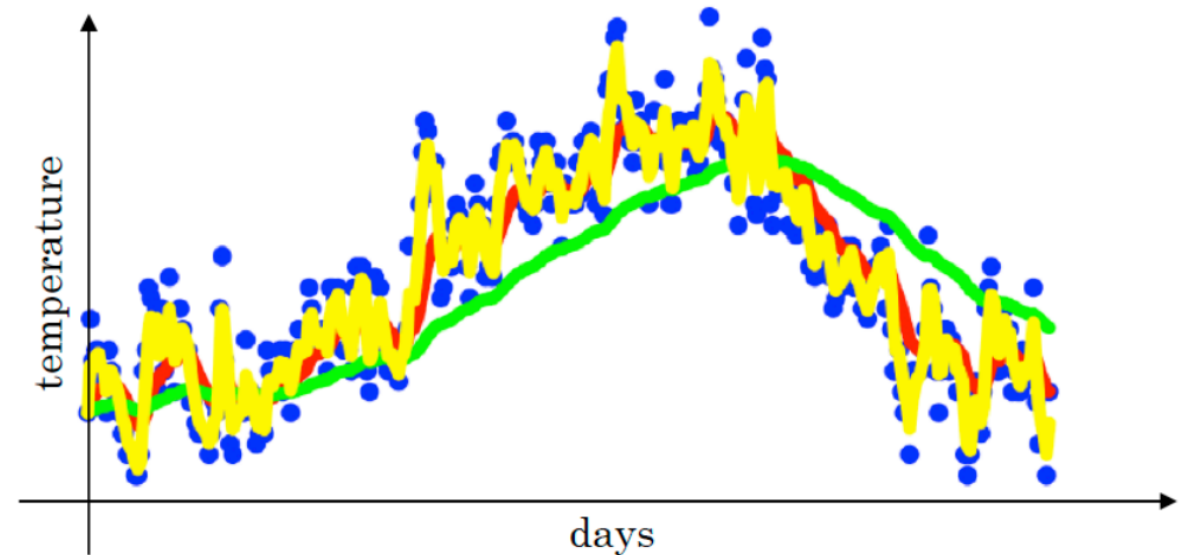
$$v_1 = 0.5v_0 + 0.5\theta_1 = 12$$

$$v_2 = 0.5v_1 + 0.5\theta_2 = 20.5$$

$$v_3 = 0.5v_2 + 0.5\theta_3 = 22.75$$

$$v_4 = 0.5v_3 + 0.5\theta_4 = 22.875$$

...



Exponentially Weighted Moving Average

- Definition: $v_t = \rho v_{t-1} + (1 - \rho)\theta_t$
- If we expand the recursive expression:

$$\begin{aligned} v_t &= (1 - \rho)\theta_t + \rho v_{t-1} \\ &= (1 - \rho)\theta_t + \rho(\rho v_{t-2} + (1 - \rho)\theta_{t-1}) \\ &= (1 - \rho)\theta_t + (1 - \rho)\rho\theta_{t-1} + \rho^2 v_{t-2} \\ &= \dots \\ &= (1 - \rho)\theta_t + (1 - \rho)\rho\theta_{t-1} + (1 - \rho)\rho^2\theta_{t-2} + \dots + (1 - \rho)\rho^{t-1}\theta_1 \end{aligned}$$

- We apply exponential decay $\rho^0, \rho^1, \rho^2 \dots$ for $\theta_t, \theta_{t-1}, \theta_{t-2} \dots$

SGD + Momentum

- **Motivation:** prevent big changes in **gradients**, try to keep some of the momentum from the past

SGD

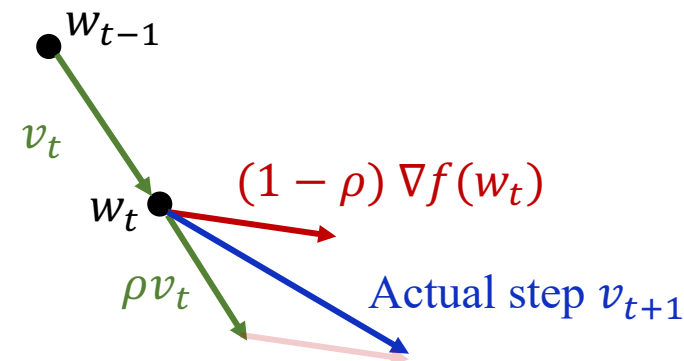
$$\mathbf{w}_{t+1} = \mathbf{w}_t - \alpha \nabla f(\mathbf{w}_t)$$

where t denotes the t -th iteration

SGD+Momentum

$$\mathbf{v}_{t+1} = \rho \mathbf{v}_t + (1 - \rho) \nabla f(\mathbf{w}_t)$$

$$\mathbf{w}_{t+1} = \mathbf{w}_t - \alpha \mathbf{v}_{t+1}$$



- Build up “velocity” v as an **exponentially weighted moving average** of gradients
- ρ controls how much we keep the previous velocity
- Typically, $\rho = 0.9$ or 0.99

RMSProp

- **Motivation:** adapting **the learning rate** with running mean of squared gradients

$$S_{t+1,w[j]} = \beta S_{t,w[j]} + (1 - \beta) \left(\nabla_{w[j]} f(\mathbf{w}_t) \right)^2$$
$$\mathbf{w}_{t+1}[j] = \mathbf{w}_t[j] - \frac{\alpha}{\sqrt{S_{t+1,w[j]} + \epsilon}} \nabla_{w[j]} f(\mathbf{w}_t)$$

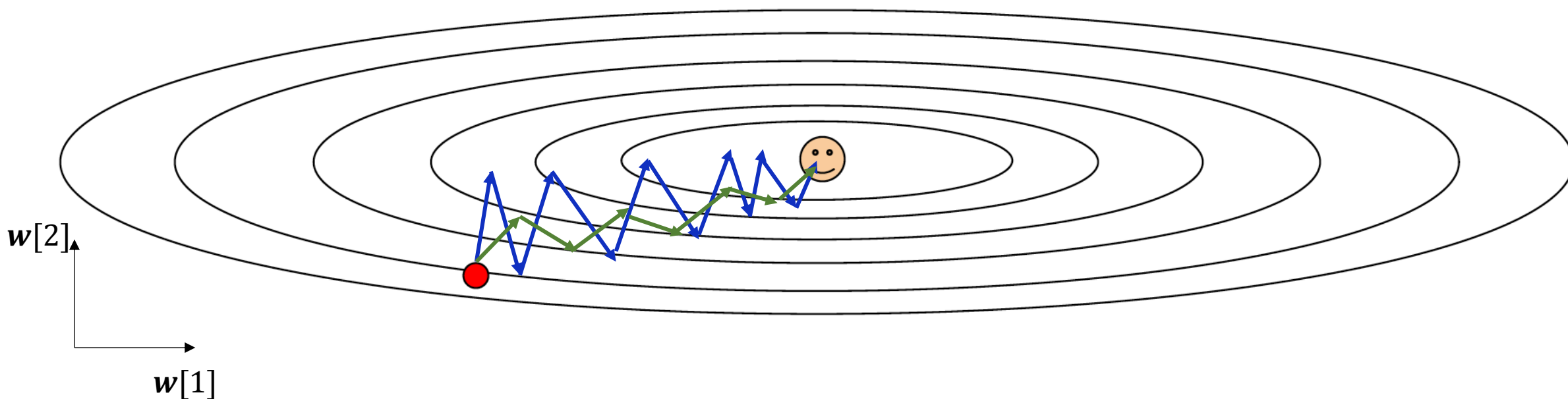
The gradient along a particular weight $\mathbf{w}[j]$

Constant to avoid zero denominator, typically 10^{-8}

- Note that the learning rate is adjusted differently for different weights $\mathbf{w}[j]$
 - The larger gradient along the weight, the slower update is made
 - The smaller gradient along the weight, the faster update is made

RMSProp

- Example:
 - Let $\mathbf{w}[2]$ be the direction that has larger gradient, and $\mathbf{w}[1]$ be the direction that has smaller gradient
 - The learning rate for $\mathbf{w}[1]$ will be enlarged, while the learning rate of $\mathbf{w}[2]$ will be decreased



Adam

- **Motivation:** combine momentum and RMSProp

SGD+Momentum

$$\mathbf{v}_{t+1} = \rho \mathbf{v}_t + (1 - \rho) \nabla f(\mathbf{w}_t)$$

$$\mathbf{w}_{t+1} = \mathbf{w}_t - \alpha \mathbf{v}_{t+1}$$

RMSProp

$$S_{t+1, \mathbf{w}[j]} = \beta S_{t, \mathbf{w}[j]} + (1 - \beta) \left(\nabla_{\mathbf{w}[j]} f(\mathbf{w}_t) \right)^2$$

$$\mathbf{w}_{t+1}[j] = \mathbf{w}_t[j] - \frac{\alpha}{\sqrt{S_{t+1, \mathbf{w}[j]} + \epsilon}} \nabla_{\mathbf{w}[j]} f(\mathbf{w}_t)$$

Adam

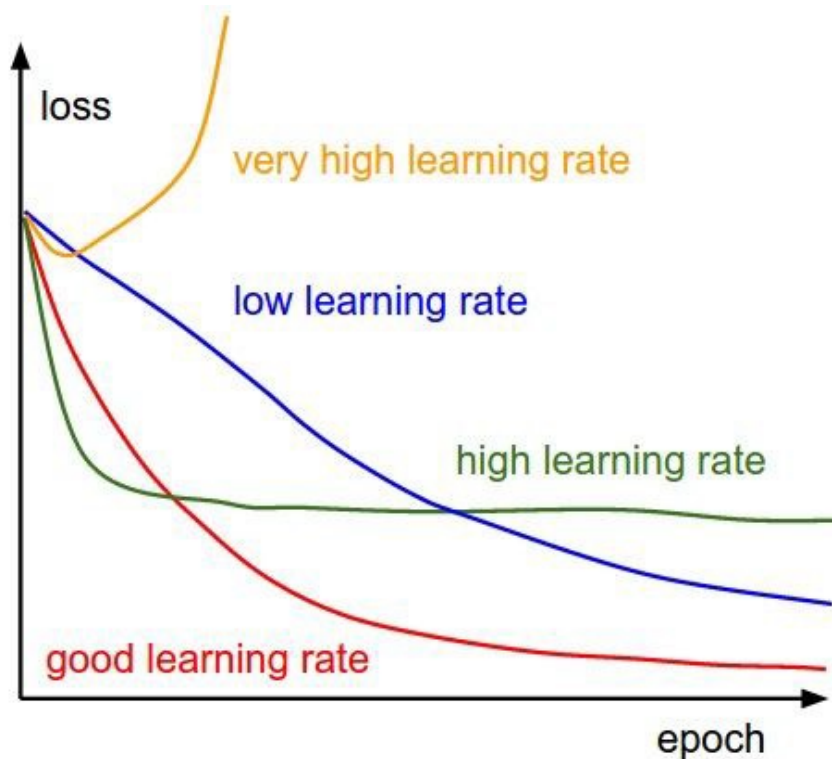
$$\mathbf{v}_{t+1} = \rho \mathbf{v}_t + (1 - \rho) \nabla f(\mathbf{w}_t)$$

$$S_{t+1, \mathbf{w}[j]} = \beta S_{t, \mathbf{w}[j]} + (1 - \beta) \left(\nabla_{\mathbf{w}[j]} f(\mathbf{w}_t) \right)^2$$

$$\mathbf{w}_{t+1}[j] = \mathbf{w}_t[j] - \frac{\alpha}{\sqrt{S_{t+1, \mathbf{w}[j]} + \epsilon}} \mathbf{v}_{t+1}[j]$$

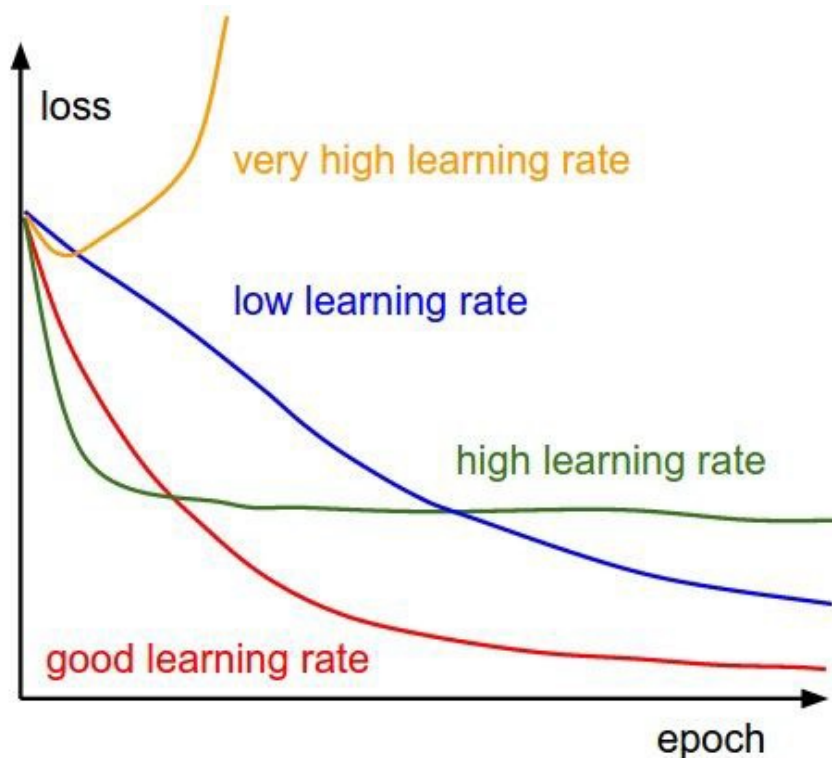
- People tend to use Adam in most cases
- You can set a higher learning rate α , the algorithm can adjust it accordingly.
- Typically converge faster

SGD, SGD+Momentum, RMSProp, Adam all have learning rate as a hyperparameter.



Q: Which one of these learning rates is best to use?

SGD, SGD+Momentum, RMSProp, Adam all have learning rate as a hyperparameter.



⇒ Learning rate decay over time!

step decay:

e.g. decay learning rate by half every few epochs.

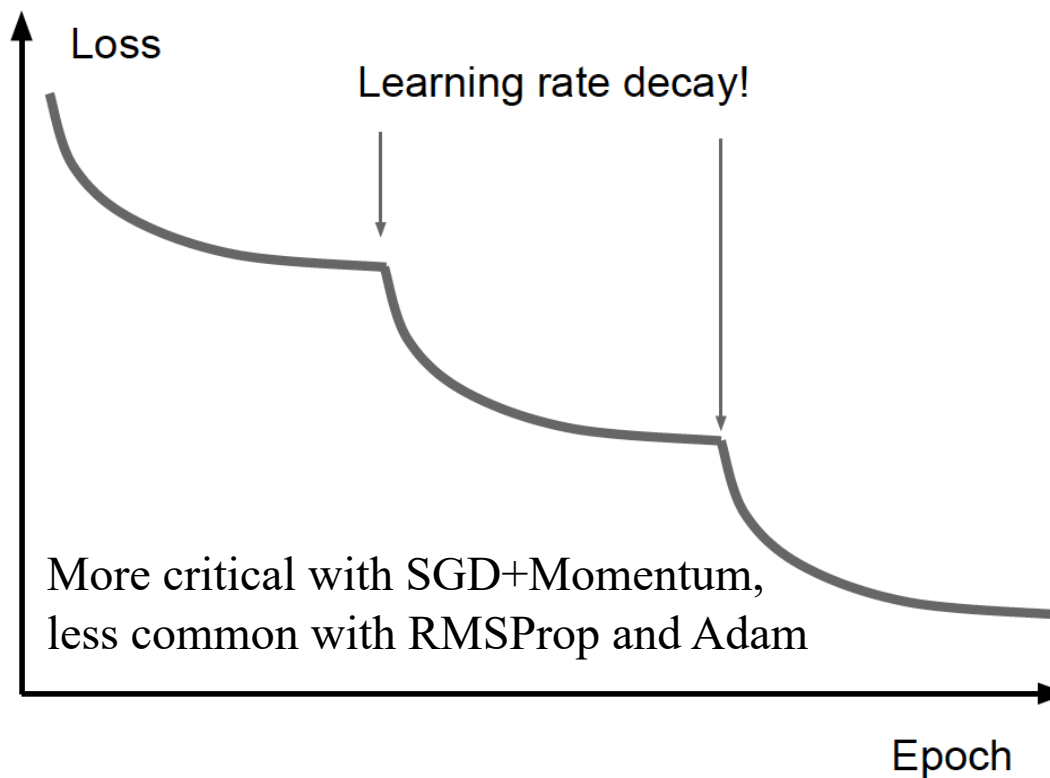
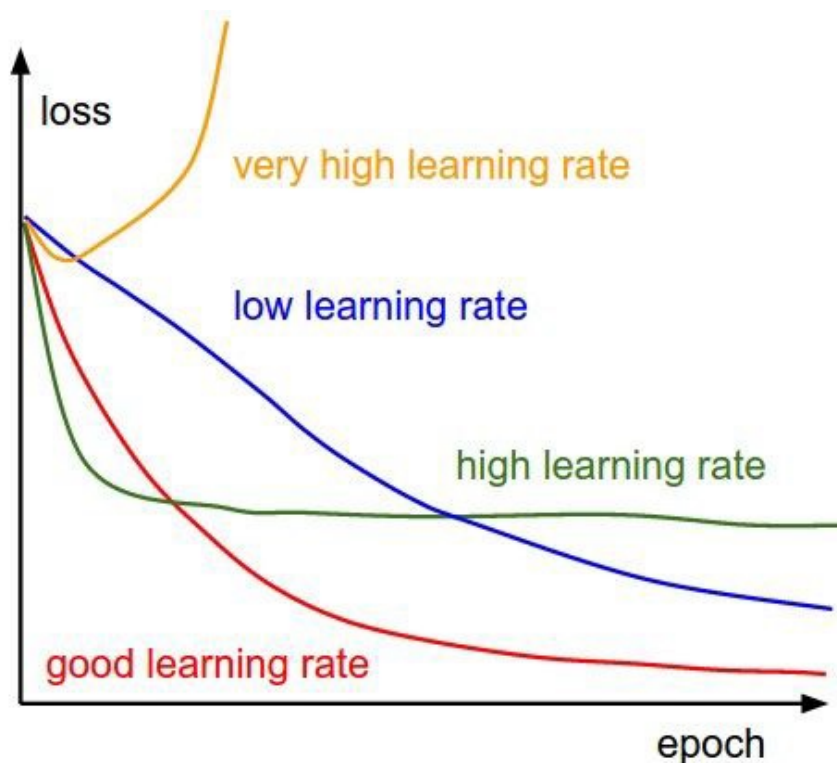
exponential decay:

$$\alpha = \alpha_0 e^{-kt}$$

1/t decay:

$$\alpha = \alpha_0 / (1 + kt)$$

SGD, SGD+Momentum, RMSProp, Adam all have learning rate as a hyperparameter.



Summary

- Neural Networks
 - Stacking non-linear layers of neurons to learn features from different levels
 - Logistic regression is a special case of NN with one single layer.
- Training neural networks
 - Applying the chain rule of derivatives → backpropagation
 - Optimization methods:
 - Gradient ascent/gradient descent
 - Batch/mini-batch/stochastic gradient ascent/descent
 - Momentum, RMSProp, Adam
- Activation functions
- Regularization – L1, L2, Dropout, Data Augmentation